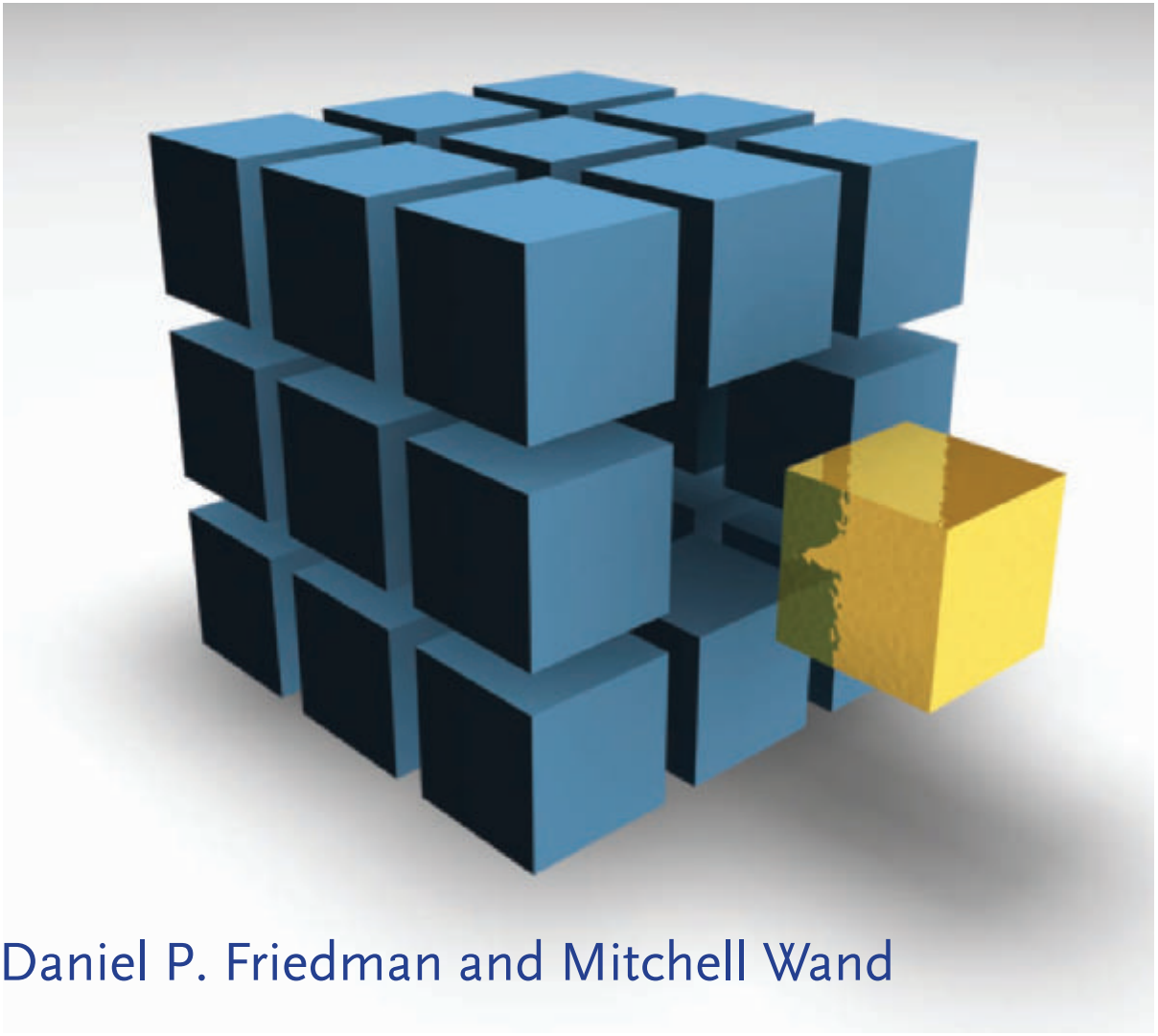
ESSENTIALS OF PROGRAMMING LANGUAGES

THIRD EDITION



Daniel P. Friedman and Mitchell Wand

Essentials of Programming Languages

third edition

Essentials of Programming Languages

third edition

Daniel P. Friedman
Mitchell Wand

The MIT Press
Cambridge, Massachusetts
London, England

MIT Press books may be purchased at special quantity discounts for business or sales promotional use. For information, please email special_sales@mitpress.mit.edu or write to Special Sales Department, The MIT Press, 55 Hayward Street, Cambridge, MA 02142.

This book was set in LaTeX 2$_\varepsilon$ by the authors, and was printed and bound in the United States of America.

Library of Congress Cataloging-in-Publication Data

Friedman, Daniel P.
Essentials of programming languages / Daniel P. Friedman, Mitchell Wand.
—3rd ed.
p. cm.
Includes bibliographical references and index.
ISBN 978-0-262-06279-4 (hbk. : alk. paper)
1. Programming Languages (Electronic computers). I. Wand, Mitchell. II. Title.

QA76.7.F73 2008
005.1—dc22 2007039723

10 9 8 7 6 5 4 3 2 1

Contents

Foreword

This book brings you face-to-face with the most fundamental idea in computer programming:

The interpreter for a computer language is just another program.

It sounds obvious, doesn't it? But the implications are profound. If you are a computational theorist, the interpreter idea recalls Gödel's discovery of the limitations of formal logical systems, Turing's concept of a universal computer, and von Neumann's basic notion of the stored-program machine. If you are a programmer, mastering the idea of an interpreter is a source of great power. It provokes a real shift in mindset, a basic change in the way you think about programming.

I did a lot of programming before I learned about interpreters, and I produced some substantial programs. One of them, for example, was a large data-entry and information-retrieval system written in PL/I. When I implemented my system, I viewed PL/I as a fixed collection of rules established by some unapproachable group of language designers. I saw my job as not to modify these rules, or even to understand them deeply, but rather to pick through the (very) large manual, selecting this or that feature to use. The notion that there was some underlying structure to the way the language was organized, and that I might want to override some of the language designers' decisions, never occurred to me. I didn't know how to create embedded sublanguages to help organize my implementation, so the entire program seemed like a large, complex mosaic, where each piece had to be carefully shaped and fitted into place, rather than a cluster of languages, where the pieces could be flexibly combined. If you don't understand interpreters, you can still write programs; you can even be a competent programmer. But you can't be a master.

There are three reasons why as a programmer you should learn about interpreters.

First, you will need at some point to implement interpreters, perhaps not interpreters for full-blown general-purpose languages, but interpreters just the same. Almost every complex computer system with which people interact in flexible ways—a computer drawing tool or an information-retrieval system, for example—includes some sort of interpreter that structures the interaction. These programs may include complex individual operations—shading a region on the display screen, or performing a database search—but the interpreter is the glue that lets you combine individual operations into useful patterns. Can you use the result of one operation as the input to another operation? Can you name a sequence of operations? Is the name local or global? Can you parameterize a sequence of operations, and give names to its inputs? And so on. No matter how complex and polished the individual operations are, it is often the quality of the glue that most directly determines the power of the system. It's easy to find examples of programs with good individual operations, but lousy glue; looking back on it, I can see that my PL/I database program certainly had lousy glue.

Second, even programs that are not themselves interpreters have important interpreter-like pieces. Look inside a sophisticated computer-aided design system and you're likely to find a geometric recognition language, a graphics interpreter, a rule-based control interpreter, and an object-oriented language interpreter all working together. One of the most powerful ways to structure a complex program is as a collection of languages, each of which provides a different perspective, a different way of working with the program elements. Choosing the right kind of language for the right purpose, and understanding the implementation tradeoffs involved: that's what the study of interpreters is about.

The third reason for learning about interpreters is that programming techniques that explicitly involve the structure of language are becoming increasingly important. Today's concern with designing and manipulating class hierarchies in object-oriented systems is only one example of this trend. Perhaps this is an inevitable consequence of the fact that our programs are becoming increasingly complex—thinking more explicitly about languages may be our best tool for dealing with this complexity. Consider again the basic idea: the interpreter itself is just a program. But that program is written in some language, whose interpreter is itself just a program written in some language whose interpreter is itself . . . Perhaps the whole distinction between program and programming language is a misleading idea, and

future programmers will see themselves not as writing programs in particular, but as creating new languages for each new application.

Friedman and Wand have done a landmark job, and their book will change the landscape of programming-language courses. They don't just *tell* you about interpreters; they *show* them to you. The core of the book is a tour de force sequence of interpreters starting with an abstract high-level language and progressively making linguistic features explicit until we reach a state machine. You can actually run this code, study and modify it, and change the way these interpreters handle scoping, parameter-passing, control structure, etc.

Having used interpreters to study the execution of languages, the authors show how the same ideas can be used to analyze programs without running them. In two new chapters, they show how to implement type checkers and inferencers, and how these features interact in modern object-oriented languages.

Part of the reason for the appeal of this approach is that the authors have chosen a good tool—the Scheme language, which combines the uniform syntax and data-abstraction capabilities of Lisp with the lexical scoping and block structure of Algol. But a powerful tool becomes most powerful in the hands of masters. The sample interpreters in this book are outstanding models. Indeed, since they are *runnable* models, I'm sure that these interpreters and analyzers will find themselves at the cores of many programming systems over the coming years.

This is not an easy book. Mastery of interpreters does not come easily, and for good reason. The language designer is a further level removed from the end user than is the ordinary application programmer. In designing an application program, you think about the specific tasks to be performed, and consider what features to include. But in designing a language, you consider the various applications people might want to implement, and the ways in which they might implement them. Should your language have static or dynamic scope, or a mixture? Should it have inheritance? Should it pass parameters by reference or by value? Should continuations be explicit or implicit? It all depends on how you expect your language to be used, which kinds of programs should be easy to write, and which you can afford to make more difficult.

Also, interpreters really *are* subtle programs. A simple change to a line of code in an interpreter can make an enormous difference in the behavior of the resulting language. Don't think that you can just skim these programs—very few people in the world can glance at a new interpreter and predict

from that how it will behave even on relatively simple programs. So study these programs. Better yet, *run* them—this is working code. Try interpreting some simple expressions, then more complex ones. Add error messages. Modify the interpreters. Design your own variations. Try to really master these programs, not just get a vague feeling for how they work.

If you do this, you will change your view of your programming, and your view of yourself as a programmer. You'll come to see yourself as a designer of languages rather than only a user of languages, as a person who chooses the rules by which languages are put together, rather than only a follower of rules that other people have chosen.

Postscript to the Third Edition

The foreword above was written only seven years ago. Since then, information applications and services have entered the lives of people around the world in ways that hardly seemed possible in 1990. They are powered by an ever—growing collection of programming languages and programming frameworks—all erected on an ever-expanding platform of interpreters.

Do you want to create Web pages? In 1990, that meant formatting static text and graphics, in effect, creating a program to be run by browsers executing only a single "print" statement. Today's dynamic Web pages make full use of scripting languages (another name for interpreted languages) like Javascript. The browser programs can be complex, and including asynchronous calls to a Web server that is typically running a program in a completely different programming framework possibly with a host of services, each with its own individual language.

Or you might be creating a bot for enhancing the performance of your avatar in a massive online multiplayer game like World of Warcraft. In that case, you're probably using a scripting language like Lua, possibly with an object-oriented extension to help in expressing classes of behaviors.

Or maybe you're programming a massive computing cluster to do indexing and searching on a global scale. If so, you might be writing your programs using the map-reduce paradigm of functional programming to relieve you of dealing explicitly with the details of how the individual processors are scheduled.

Or perhaps you're developing new algorithms for sensor networks, and exploring the use of lazy evaluation to better deal with parallelism and data aggregation. Or exploring transformation systems like XSLT for controlling Web pages. Or designing frameworks for transforming and remixing multimedia streams. Or . . .

So many new applications! So many new languages! So many new interpreters!

As ever, novice programmers, even capable ones, can get along viewing each new framework individually, working within its fixed set of rules. But creating new frameworks requires skills of the master: understanding the principles that run across languages, appreciating which language features are best suited for which type of application, and knowing how to craft the interpreters that bring these languages to life. These are the skills you will learn from this book.

Hal Abelson
Cambridge, Massachusetts
September 2007

Preface

Goal

This book is an analytic study of programming languages. Our goal is to provide a deep, working understanding of the essential concepts of programming languages. These essentials have proved to be of enduring importance; they form a basis for understanding future developments in programming languages.

Most of these essentials relate to the semantics, or meaning, of program elements. Such meanings reflect how program elements are interpreted as the program executes. Programs called interpreters provide the most direct, executable expression of program semantics. They process a program by directly analyzing an abstract representation of the program text. We therefore choose interpreters as our primary vehicle for expressing the semantics of programming language elements.

The most interesting question about a program as object is, "What does it do?" The study of interpreters tells us this. Interpreters are critical because they reveal nuances of meaning, and are the direct path to more efficient compilation and to other kinds of program analyses.

Interpreters are also illustrative of a broad class of systems that transform information from one form to another based on syntax structure. Compilers, for example, transform programs into forms suitable for interpretation by hardware or virtual machines. Though general compilation techniques are beyond the scope of this book, we do develop several elementary program translation systems. These reflect forms of program analysis typical of compilation, such as control transformation, variable binding resolution, and type checking.

The following are some of the strategies that distinguish our approach.

1. Each new concept is explained through the use of a small language. These languages are often cumulative: later languages may rely on the features of earlier ones.

2. Language processors such as interpreters and type checkers are used to explain the behavior of programs in a given language. They express language design decisions in a manner that is both formal (unambiguous and complete) and executable.

3. When appropriate, we use interfaces and specifications to create data abstractions. In this way, we can change data representation without changing programs. We use this to investigate alternative implementation strategies.

4. Our language processors are written both at the very high level needed to produce a concise and comprehensible view of semantics and at the much lower level needed to understand implementation strategies.

5. We show how simple algebraic manipulation can be used to predict the behavior of programs and to derive their properties. In general, however, we make little use of mathematical notation, preferring instead to study the behavior of programs that constitute the implementations of our languages.

6. The text explains the key concepts, while the exercises explore alternative designs and other issues. For example, the text deals with static binding, but dynamic binding is discussed in the exercises. One thread of exercises applies the concept of lexical addressing to the various languages developed in the book.

We provide several views of programming languages using widely varying levels of abstraction. Frequently our interpreters provide a very high-level view that expresses language semantics in a very concise fashion, not far from that of formal mathematical semantics. At the other extreme, we demonstrate how programs may be transformed into a very low-level form characteristic of assembly language. By accomplishing this transformation in small stages, we maintain a clear connection between the high-level and low-level views.

We have made some significant changes to this edition. We have included informal contracts with all nontrivial definitions. This has the effect of clarifying the chosen abstractions. In addition, the chapter on modules is completely new. To make implementations simpler, the source language for chapters 3, 4, 5, 7, and 8 assumes that exactly one argument can be passed to a function; we have included exercises that support multiargument procedures. Chapter 6 is completely new, since we have opted for a first-order compositional continuation-passing-style transform rather than a relational one. Also, because of the nature of tail-form expressions, we use multiargument procedures here, and in the objects and classes chapter, we do the same, though there it is not so necessary. Every chapter has been revised and many new exercises have been added.

Organization

The first two chapters provide the foundations for a careful study of programming languages. Chapter 1 emphasizes the connection between inductive data specification and recursive programming and introduces several notions related to the scope of variables. Chapter 2 introduces a data type facility. This leads to a discussion of data abstraction and examples of representational transformations of the sort used in subsequent chapters.

Chapter 3 uses these foundations to describe the behavior of programming languages. It introduces interpreters as mechanisms for explaining the runtime behavior of languages and develops an interpreter for a simple, lexically scoped language with first-class procedures and recursion. This interpreter is the basis for much of the material in the remainder of the book. The chapter ends by giving a thorough treatment of a language that uses indices in place of variables and as a result variable lookup can be via a list reference.

Chapter 4 introduces a new component, the state, which maps locations to values. Once this is added, we can look at various questions of representation. In addition, it permits us to explore call-by-reference, call-by-name, and call-by-need parameter-passing mechanisms.

Chapter 5 rewrites our basic interpreter in continuation-passing style. The control structure that is needed to run the interpreter thereby shifts from recursion to iteration. This exposes the control mechanisms of the interpreted language, and strengthens one's intuition for control issues in general. It also allows us to extend the language with trampolining, exception-handling, and multithreading mechanisms.

Chapter 6 is the companion to the previous chapter. There we show how to transform our familiar interpreter into continuation-passing style; here we show how to accomplish this for a much larger class of programs. Continuation-passing style is a powerful programming tool, for it allows any sequential control mechanism to be implemented in almost any language. The algorithm is also an example of an abstractly specified source-to-source program transformation.

Chapter 7 turns the language of chapter 3 into a typed language. First we implement a type checker. Then we show how the types in a program can be deduced by a unification-based type inference algorithm.

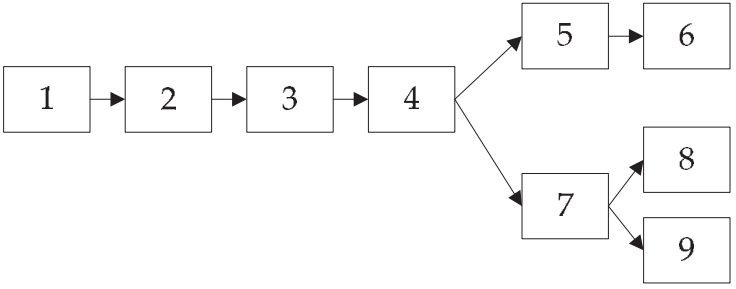
Chapter 8 builds typed modules relying heavily on an understanding of the previous chapter. Modules allow us to build and enforce abstraction boundaries, and they offer a new kind of scoping.

Chapter 9 presents the basic concepts of object-oriented languages, centered on classes. We first develop an efficient run-time architecture, which is used as the basis for the material in the second part of the chapter. The second part combines the ideas of the type checker of chapter 7 with those of the object-oriented language of the first part, leading to a conventional typed object-oriented language. This requires introducing new concepts including interfaces, abstract methods, and casting.

For Further Reading explains where each of the ideas in the book has come from. This is a personal walk-through allowing the reader the opportunity to visit each topic from the original paper, though in some cases, we have just chosen an accessible source.

Finally, appendix B describes our SLLGEN parsing system.

The dependencies of the various chapters are shown in the figure below.



Usage

This material has been used in both undergraduate and graduate courses. Also, it has been used in continuing education courses for professional programmers. We assume background in data structures and experience both in a procedural language such as C, C++, or Java, and in Scheme, ML, Python, or Haskell.

Exercises are a vital part of the text and are scattered throughout. They range in difficulty from being trivial if related material is understood [⋆], to requiring many hours of thought and programming work [⋆ ⋆ ⋆]. A great deal of material of applied, historical, and theoretical interest resides within them. We recommend that each exercise be read and some thought be given as to how to solve it. Although we write our program interpretation and transformation systems in Scheme, any language that supports both first-class procedures and assignment (ML, Common Lisp, Python, Ruby, etc.) is adequate for working the exercises.

Exercise 0.1 [⋆] We often use phrases like "some languages have property *X*." For each such phrase, find one or more languages that have the property and one or more languages that do not have the property. Feel free to ferret out this information from any descriptive book on programming languages (say Scott (2005), Sebesta (2007), or Pratt & Zelkowitz (2001)).

This is a hands-on book: everything discussed in the book may be implemented within the limits of a typical university course. Because the abstraction facilities of functional programming languages are especially suited to this sort of programming, we can write substantial language-processing systems that are nevertheless compact enough that one can understand and manipulate them with reasonable effort.

The web site, available through the publisher, includes complete Scheme code for all of the interpreters and analyzers in this book. The code is written in PLT Scheme. We chose this Scheme implementation because its module system and programming environment provide a substantial advantage to the student. The code is largely R^5RS-compatible, and should be easily portable to any full-featured Scheme implementation.

Acknowledgments

We are indebted to countless colleagues and students who used and critiqued the first two editions of this book and provided invaluable assistance in the long gestation of this third edition. We are especially grateful for the contributions of the following individuals, to whom we offer a special word of thanks. Olivier Danvy encouraged our consideration of a first-order compositional continuation-passing algorithm and proposed some interesting exercises. Matthias Felleisen's keen analysis has improved the design of several chapters. Amr Sabry made many useful suggestions and found at least one extremely subtle bug in a draft of chapter 9. Benjamin Pierce offered a number of insightful observations after teaching from the first edition, almost all of which we have incorporated. Gary Leavens provided exceptionally thorough and valuable comments on early drafts of the second edition, including a large number of detailed suggestions for change. Stephanie Weirich found a subtle bug in the type inference code of the second edition of chapter 7. Ryan Newton, in addition to reading a draft of the second edition, assumed the onerous task of suggesting a difficulty level for each exercise for that edition. Chung-chieh Shan taught from an early draft of the third edition and provided copious and useful comments.

Kevin Millikin, Arthur Lee, Roger Kirchner, Max Hailperin, and Erik Hilsdale all used early drafts of the second edition. Will Clinger, Will Byrd, Joe Near, and Kyle Blocher all used drafts of this edition. Their comments have been extremely valuable. Ron Garcia, Matthew Flatt, Shriram Krishnamurthi, Steve Ganz, Gregor Kiczales, Marlene Miller, Galen Williamson, Dipanwita Sarkar, Steven Bogaerts, Albert Rossi, Craig Citro, Christopher Dutchyn, Jeremy Siek, and Neil Ching also provided careful reading and useful comments.

Several people deserve special thanks for assisting us with this book. We want to thank Neil Ching for developing the index. Jonathan Sobel and Erik Hilsdale built several prototype implementations and contributed many ideas as we experimented with the design of the `define-datatype` and `cases` syntactic extensions. The Programming Language Team, and especially Matthias Felleisen, Matthew Flatt, Robby Findler, and Shriram Krishnamurthi, were very helpful in providing compatibility with their DrScheme system. Kent Dybvig developed the exceptionally efficient and robust Chez Scheme implementation, which the authors have used for decades. Will Byrd has provided invaluable assistance during the entire process. Matthias Felleisen strongly urged us to adopt compatibility with DrScheme's module system, which is evident in the implementation that can be found at `http://mitpress.mit.edu/eopl3`.

Some have earned special mention for their thoughtfulness and concern for our well-being. George Springer and Larry Finkelstein have each supplied invaluable support. Bob Prior, our wonderful editor at MIT Press, deserves special thanks for his encouragement in getting us to attack the writing of this edition. Ada Brunstein, Bob's successor, also deserves thanks for making our transition to a new editor so smoothly. Indiana University's School of Informatics and Northeastern University's College of Computer and Information Science have created an environment that has allowed us to undertake this project. Mary Friedman's gracious hosting of several week-long writing sessions did much to accelerate our progress.

We want to thank Christopher T. Haynes for his collaboration on the first two editions. Unfortunately, his interests have shifted elsewhere, and he has not continued with us on this edition.

Finally, we are most grateful to our families for tolerating our passion for working on the book. Thank you Rob, Shannon, Rachel, Sara, and Mary; and thank you Rebecca and Joshua, Jennifer and Stephen, Joshua and Georgia, and Barbara.

This edition has been in the works for a while and we have likely overlooked someone who has helped along the way. We regret any oversight. You see this written in books all the time and wonder why anyone would write it. Of course, you regret any oversight. But, when you have an army of helpers (it takes a village), you really feel a sense of obligation not to forget anyone. So, if you were overlooked, we are truly sorry.

— D.P.F. and M.W.

1 *Inductive Sets of Data*

This chapter introduces the basic programming tools we will need to write interpreters, checkers and similar programs that form the heart of a programming language processor.

Because the syntax of a program in a language is usually a nested or tree-like structure, recursion will be at the core of our techniques. Section 1.1 and section 1.2 introduce methods for inductively specifying data structures and show how such specifications may be used to guide the construction of recursive programs. Section 1.3 shows how to extend these techniques to more complex problems. The chapter concludes with an extensive set of exercises. These exercises are the heart of this chapter. They provide experience that is essential for mastering the technique of recursive programming upon which the rest of this book is based.

1.1 Recursively Specified Data

When writing code for a procedure, we must know precisely what kinds of values may occur as arguments to the procedure, and what kinds of values are legal for the procedure to return. Often these sets of values are complex. In this section we introduce formal techniques for specifying sets of values.

1.1.1 Inductive Specification

Inductive specification is a powerful method of specifying a set of values. To illustrate this method, we use it to describe a certain subset S of the natural numbers $N = \{0, 1, 2, \ldots\}$.

Definition 1.1.1 *A natural number n is in S if and only if*

1. $n = 0$, *or*
2. $n - 3 \in S$.

Let us see how we can use this definition to determine what natural numbers are in S. We know that $0 \in S$. Therefore $3 \in S$, since $(3 - 3) = 0$ and $0 \in S$. Similarly $6 \in S$, since $(6 - 3) = 3$ and $3 \in S$. Continuing in this way, we can conclude that all multiples of 3 are in S.

What about other natural numbers? Is $1 \in S$? We know that $1 \neq 0$, so the first condition is not satisfied. Furthermore, $(1 - 3) = -2$, which is not a natural number and thus is not a member of S. Therefore the second condition is not satisfied. Since 1 satisfies neither condition, $1 \notin S$. Similarly, $2 \notin S$. What about 4? $4 \in S$ only if $1 \in S$. But $1 \notin S$, so $4 \notin S$, as well. Similarly, we can conclude that if n is a natural number and is not a multiple of 3, then $n \notin S$.

From this argument, we conclude that S is the set of natural numbers that are multiples of 3.

We can use this definition to write a procedure to decide whether a natural number n is in S.

```
in-S? : N → Bool
usage: (in-S? n) = #t if n is in S, #f otherwise
(define in-S?
  (lambda (n)
    (if (zero? n) #t
      (if (>= (- n 3) 0)
        (in-S? (- n 3))
        #f))))
```

Here we have written a recursive procedure in Scheme that follows the definition. The notation **in-S?** : $N \to$ *Bool* is a comment, called the *contract* for this procedure. It means that `in-S?` is intended to be a procedure that takes a natural number and produces a boolean. Such comments are helpful for reading and writing code.

To determine whether $n \in S$, we first ask whether $n = 0$. If it is, then the answer is true. Otherwise we need to see whether $n - 3 \in S$. To do this, we first check to see whether $(n - 3) \geq 0$. If it is, we then can use our procedure to see whether it is in S. If it is not, then n cannot be in S.

Here is an alternative way of writing down the definition of S.

Definition 1.1.2 *Define the set S to be the smallest set contained in N and satisfying the following two properties:*

1. $0 \in S$, *and*

2. *if* $n \in S$, *then* $n + 3 \in S$.

A "smallest set" is the one that satisfies properties 1 and 2 and that is a subset of any other set satisfying properties 1 and 2. It is easy to see that there can be only one such set: if S_1 and S_2 both satisfy properties 1 and 2, and both are smallest, then $S_1 \subseteq S_2$ (since S_1 is smallest), and $S_2 \subseteq S_1$ (since S_2 is smallest), hence $S_1 = S_2$. We need this extra condition, because otherwise there are many sets that satisfy the remaining two conditions (see exercise 1.3).

Here is yet another way of writing the definition:

$$\frac{}{0 \in S}$$

$$\frac{n \in S}{(n+3) \in S}$$

This is simply a shorthand notation for the preceding version of the definition. Each entry is called a *rule of inference*, or just a *rule*; the horizontal line is read as an "if-then." The part above the line is called the *hypothesis* or the *antecedent*; the part below the line is called the *conclusion* or the *consequent*. When there are two or more hypotheses listed, they are connected by an implicit "and" (see definition 1.1.5). A rule with no hypotheses is called an *axiom*. We often write an axiom without the horizontal line, like

$$0 \in S$$

The rules are interpreted as saying that a natural number n is in S if and only if the statement "$n \in S$" can be derived from the axioms by using the rules of inference finitely many times. This interpretation automatically makes S the smallest set that is closed under the rules.

These definitions all say the same thing. We call the first version a *top-down* definition, the second version a *bottom-up* definition, and the third version a *rules-of-inference* version.

Let us see how this works on some other examples.

Definition 1.1.3 (list of integers, top-down) *A Scheme list is a* list of integers *if and only if either*

1. *it is the empty list, or*
2. *it is a pair whose car is an integer and whose cdr is a list of integers.*

We use *Int* to denote the set of all integers, and *List-of-Int* to denote the set of lists of integers.

Definition 1.1.4 (list of integers, bottom-up) *The set List-of-Int is the smallest set of Scheme lists satisfying the following two properties:*

1. `()` $\in$ *List-of-Int, and*
2. *if* $n \in$ *Int and* $l \in$ *List-of-Int, then* `(`n `.` l`)` $\in$ *List-of-Int.*

Here we use the infix "`.`" to denote the result of the `cons` operation in Scheme. The phrase `(`n `.` l`)` denotes a Scheme pair whose car is n and whose cdr is l.

Definition 1.1.5 (list of integers, rules of inference)

$$() \in \textit{List-of-Int}$$

$$\frac{n \in \textit{Int} \qquad l \in \textit{List-of-Int}}{(n\ .\ l) \in \textit{List-of-Int}}$$

These three definitions are equivalent. We can show how to use them to generate some elements of *List-of-Int*.

1. `()` is a list of integers, because of property 1 of definition 1.1.4 or the first rule of definition 1.1.5.
2. `(14 . ())` is a list of integers, because of property 2 of definition 1.1.4, since `14` is an integer and `()` is a list of integers. We can also write this as an instance of the second rule for *List-of-Int*.

$$\frac{\texttt{14} \in \textit{Int} \qquad \texttt{()} \in \textit{List-of-Int}}{\texttt{(14 . ())} \in \textit{List-of-Int}}$$

3. `(3 . (14 . ()))` is a list of integers, because of property 2, since `3` is an integer and `(14 . ())` is a list of integers. We can write this as another instance of the second rule for *List-of-Int*.

$$\frac{\texttt{3} \in \mathit{Int} \qquad \texttt{(14 . ())} \in \mathit{List\text{-}of\text{-}Int}}{\texttt{(3 . (14 . ()))} \in \mathit{List\text{-}of\text{-}Int}}$$

4. `(-7 . (3 . (14 . ())))` is a list of integers, because of property 2, since `-7` is a integer and `(3 . (14 . ()))` is a list of integers. Once more we can write this as an instance of the second rule for *List-of-Int*.

$$\frac{\texttt{-7} \in \mathit{Int} \qquad \texttt{(3 . (14 . ()))} \in \mathit{List\text{-}of\text{-}Int}}{\texttt{(-7 . (3 . (14 . ())))} \in \mathit{List\text{-}of\text{-}Int}}$$

5. Nothing is a list of integers unless it is built in this fashion.

Converting from dot notation to list notation, we see that `()`, `(14)`, `(3 14)`, and `(-7 3 14)` are all members of *List-of-Int*.

We can also combine the rules to get a picture of the entire chain of reasoning that shows that `(-7 . (3 . (14 . ())))` $\in$ *List-of-Int*. The tree-like picture below is called a *derivation* or *deduction tree*.

$$\dfrac{\texttt{-7} \in N \qquad \dfrac{\texttt{3} \in N \qquad \dfrac{\texttt{14} \in N \qquad () \in \mathit{List\text{-}of\text{-}Int}}{\texttt{(14 . ())} \in \mathit{List\text{-}of\text{-}Int}}}{\texttt{(3 . (14 . ()))} \in \mathit{List\text{-}of\text{-}Int}}}{\texttt{(-7 . (3 . (14 . ())))} \in \mathit{List\text{-}of\text{-}Int}}$$

Exercise 1.1 [⋆] Write inductive definitions of the following sets. Write each definition in all three styles (top-down, bottom-up, and rules of inference). Using your rules, show the derivation of some sample elements of each set.

1. $\{3n+2 \mid n \in N\}$
2. $\{2n+3m+1 \mid n, m \in N\}$
3. $\{(n, 2n+1) \mid n \in N\}$
4. $\{(n, n^2) \mid n \in N\}$ Do not mention squaring in your rules. As a hint, remember the equation $(n+1)^2 = n^2 + 2n + 1$.

Exercise 1.2 [⋆⋆] What sets are defined by the following pairs of rules? Explain why.

1. $(0,1) \in S \qquad \dfrac{(n,k) \in S}{(n+1, k+7) \in S}$

2. $(0,1) \in S \quad \dfrac{(n,k) \in S}{(n+1,2k) \in S}$

3. $(0,0,1) \in S \quad \dfrac{(n,i,j) \in S}{(n+1,j,i+j) \in S}$

4. [⋆⋆⋆] $(0,1,0) \in S \quad \dfrac{(n,i,j) \in S}{(n+1,i+2,i+j) \in S}$

Exercise 1.3 [⋆] Find a set T of natural numbers such that $0 \in T$, and whenever $n \in T$, then $n+3 \in T$, but $T \neq S$, where S is the set defined in definition 1.1.2.

1.1.2 Defining Sets Using Grammars

The previous examples have been fairly straightforward, but it is easy to imagine how the process of describing more complex data types becomes quite cumbersome. To help with this, we show how to specify sets with *grammars*. Grammars are typically used to specify sets of strings, but we can use them to define sets of values as well.

For example, we can define the set *List-of-Int* by the grammar

List-of-Int ::= `(` `)`
List-of-Int ::= `(` *Int* `.` *List-of-Int* `)`

Here we have two rules corresponding to the two properties in definition 1.1.4 above. The first rule says that the empty list is in *List-of-Int*, and the second says that if n is in *Int* and l is in *List-of-Int*, then `(`n `.` l`)` is in *List-of-Int*. This set of rules is called a *grammar*.

Let us look at the pieces of this definition. In this definition we have

- **Nonterminal Symbols**. These are the names of the sets being defined. In this case there is only one such set, but in general, there might be several sets being defined. These sets are sometimes called *syntactic categories*.

  We will use the convention that nonterminals and sets have names that are capitalized, but we will use lower-case names when referring to their elements in prose. This is simpler than it sounds. For example, *Expression* is a nonterminal, but we will write $e \in$ *Expression* or "e is an expression."

  Another common convention, called *Backus-Naur Form* or *BNF*, is to surround the word with angle brackets, e.g. ⟨expression⟩.

- **Terminal Symbols**. These are the characters in the external representation, in this case `.`, `(`, and `)`. We typically write these using a typewriter font, e.g. `lambda`.

- **Productions**. The rules are called *productions*. Each production has a left-hand side, which is a nonterminal symbol, and a right-hand side, which consists of terminal and nonterminal symbols. The left- and right-hand sides are usually separated by the symbol ::=, read *is* or *can be*. The right-hand side specifies a method for constructing members of the syntactic category in terms of other syntactic categories and *terminal symbols*, such as the left parenthesis, right parenthesis, and the period.

Often some syntactic categories mentioned in a production are left undefined when their meaning is sufficiently clear from context, such as *Int*.

Grammars are often written using some notational shortcuts. It is common to omit the left-hand side of a production when it is the same as the left-hand side of the preceding production. Using this convention our example would be written as

$$\begin{aligned} \textit{List-of-Int} &::= \texttt{(}\ \texttt{)} \\ &::= \texttt{(}\textit{Int}\ \texttt{.}\ \textit{List-of-Int}\texttt{)} \end{aligned}$$

One can also write a set of rules for a single syntactic category by writing the left-hand side and ::= just once, followed by all the right-hand sides separated by the special symbol "|" (vertical bar, read *or*). The grammar for *List-of-Int* could be written using "|" as

$$\textit{List-of-Int} ::= \texttt{(}\ \texttt{)}\ \mid\ \texttt{(}\textit{Int}\ \ \texttt{.}\ \ \textit{List-of-Int}\texttt{)}$$

Another shortcut is the *Kleene star*, expressed by the notation $\{\ldots\}^*$. When this appears in a right-hand side, it indicates a sequence of any number of instances of whatever appears between the braces. Using the Kleene star, the definition of *List-of-Int* is simply

$$\textit{List-of-Int} ::= \texttt{(}\{\textit{Int}\}^*\texttt{)}$$

This includes the possibility of no instances at all. If there are zero instances, we get the empty string.

A variant of the star notation is *Kleene plus* $\{\ldots\}^+$, which indicates a sequence of *one* or more instances. Substituting $^+$ for * in the example above would define the syntactic category of non-empty lists of integers.

Still another variant of the star notation is the *separated list* notation. For example, we write $\{\textit{Int}\}^{*(c)}$ to denote a sequence of any number of instances of the nonterminal *Int*, separated by the non-empty character sequence *c*. This includes the possibility of no instances at all. If there are zero instances, we get the empty string. For example, $\{\textit{Int}\}^{*(,)}$ includes the strings

```
8
14, 12
7, 3, 14, 16
```

and $\{Int\}^{*(;)}$ includes the strings

```
8
14; 12
7; 3; 14; 16
```

These notational shortcuts are not essential. It is always possible to rewrite the grammar without them.

If a set is specified by a grammar, a *syntactic derivation* may be used to show that a given data value is a member of the set. Such a derivation starts with the nonterminal corresponding to the set. At each step, indicated by an arrow $\Rightarrow$, a nonterminal is replaced by the right-hand side of a corresponding rule, or with a known member of its syntactic class if the class was left undefined. For example, the previous demonstration that `(14 . ())` is a list of integers may be formalized with the syntactic derivation

List-of-Int
$\Rightarrow$ `(`*Int* `.` *List-of-Int*`)`
$\Rightarrow$ `(14 .` *List-of-Int*`)`
$\Rightarrow$ `(14 . ())`

The order in which nonterminals are replaced does not matter. Thus, here is another derivation of `(14 . ())`.

List-of-Int
$\Rightarrow$ `(`*Int* `.` *List-of-Int*`)`
$\Rightarrow$ `(`*Int* `. ())`
$\Rightarrow$ `(14 . ())`

Exercise 1.4 [⋆] Write a derivation from *List-of-Int* to `(-7 . (3 . (14 . ())))`.

Let us consider the definitions of some other useful sets.

1. Many symbol manipulation procedures are designed to operate on lists that contain only symbols and other similarly restricted lists. We call these lists *s-lists*, defined as follows:

   Definition 1.1.6 (s-list, s-exp)

   S-list ::= `(`{*S-exp*}*`)`
   S-exp ::= *Symbol* | *S-list*

An s-list is a list of s-exps, and an s-exp is either an s-list or a symbol. Here are some s-lists.

```
(a b c)
(an (((s-list)) (with () lots) ((of) nesting)))
```

We may occasionally use an expanded definition of s-list with integers allowed, as well as symbols.

2. A binary tree with numeric leaves and interior nodes labeled with symbols may be represented using three-element lists for the interior nodes by the grammar:

Definition 1.1.7 (binary tree)

$$\textit{Bintree} ::= \textit{Int} \mid (\textit{Symbol}\ \ \textit{Bintree}\ \ \textit{Bintree})$$

Here are some examples of such trees:

```
1
2
(foo 1 2)
(bar 1 (foo 1 2))
(baz
  (bar 1 (foo 1 2))
  (biz 4 5))
```

3. The *lambda calculus* is a simple language that is often used to study the theory of programming languages. This language consists only of variable references, procedures that take a single argument, and procedure calls. We can define it with the grammar:

Definition 1.1.8 (lambda expression)

$$\begin{aligned}\textit{LcExp} &::= \textit{Identifier}\\ &::= (\texttt{lambda}\ (\textit{Identifier})\ \ \textit{LcExp})\\ &::= (\textit{LcExp}\ \ \textit{LcExp})\end{aligned}$$

where an identifier is any symbol other than `lambda`.

The identifier in the second production is the name of a variable in the body of the `lambda` expression. This variable is called the *bound variable* of the expression, because it binds or captures any occurrences of the variable in the body. Any occurrence of that variable in the body refers to this one.

To see how this works, consider the lambda calculus extended with arithmetic operators. In that language,

```
(lambda (x) (+ x 5))
```

is an expression in which `x` is the bound variable. This expression describes a procedure that adds 5 to its argument. Therefore, in

```
((lambda (x) (+ x 5)) (- x 7))
```

the last occurrence of `x` does not refer to the `x` that is bound in the `lambda` expression. We discuss this in section 1.2.4, where we introduce `occurs-free?`.

This grammar defines the elements of *LcExp* as Scheme values, so it becomes easy to write programs that manipulate them.

These grammars are said to be *context-free* because a rule defining a given syntactic category may be applied in any context that makes reference to that syntactic category. Sometimes this is not restrictive enough. Consider binary search trees. A node in a binary search tree is either empty or contains an integer and two subtrees

Binary-search-tree ::= `()` | (*Int* *Binary-search-tree* *Binary-search-tree*)

This correctly describes the structure of each node but ignores an important fact about binary search trees: all the keys in the left subtree are less than (or equal to) the key in the current node, and all the keys in the right subtree are greater than the key in the current node.

Because of this additional constraint, not every syntactic derivation from *Binary-search-tree* leads to a correct binary search tree. To determine whether a particular production can be applied in a particular syntactic derivation, we have to look at the context in which the production is applied. Such constraints are called *context-sensitive constraints* or *invariants*.

Context-sensitive constraints also arise when specifying the syntax of programming languages. For instance, in many languages every variable must be declared before it is used. This constraint on the use of variables is sensitive to the context of their use. Formal methods can be used to specify context-sensitive constraints, but these methods are far more complicated than the ones we consider in this chapter. In practice, the usual approach is first to specify a context-free grammar. Context-sensitive constraints are then added using other methods. We show an example of such techniques in chapter 7.

1.1.3 Induction

Having described sets inductively, we can use the inductive definitions in two ways: to prove theorems about members of the set and to write programs that manipulate them. Here we present an example of such a proof; writing the programs is the subject of the next section.

Theorem 1.1.1 *Let t be a binary tree, as defined in definition 1.1.7. Then t contains an odd number of nodes.*

Proof: The proof is by induction on the size of t, where we take the size of t to be the number of nodes in t. The induction hypothesis, $IH(k)$, is that any tree of size $\leq k$ has an odd number of nodes. We follow the usual prescription for an inductive proof: we first prove that $IH(0)$ is true, and we then prove that whenever k is an integer such that IH is true for k, then IH is true for $k+1$ also.

1. There are no trees with 0 nodes, so $IH(0)$ holds trivially.
2. Let k be an integer such that $IH(k)$ holds, that is, any tree with $\leq k$ nodes actually has an odd number of nodes. We need to show that $IH(k+1)$ holds as well: that any tree with $\leq k+1$ nodes has an odd number of nodes. If t has $\leq k+1$ nodes, there are exactly two possibilities according to the definition of a binary tree:
   (a) t could be of the form n, where n is an integer. In this case, t has exactly one node, and one is odd.
   (b) t could be of the form $(sym\ \ t_1\ \ t_2)$, where *sym* is a symbol and t_1 and t_2 are trees. Now t_1 and t_2 must have fewer nodes than t. Since t has $\leq k+1$ nodes, t_1 and t_2 must have $\leq k$ nodes. Therefore they are covered

by $IH(k)$, and they must each have an odd number of nodes, say $2n_1+1$ and $2n_2+1$ nodes, respectively. Hence the total number of nodes in the tree, counting the two subtrees and the root, is

$$(2n_1+1)+(2n_2+1)+1 = 2(n_1+n_2+1)+1$$

which is once again odd.

This completes the proof of the claim that $IH(k+1)$ holds and therefore completes the induction. □

The key to the proof is that the substructures of a tree t are always smaller than t itself. This pattern of proof is called *structural induction*.

Proof by Structural Induction

To prove that a proposition $IH(s)$ is true for all structures s, prove the following:

1. *IH is true on simple structures (those without substructures).*
2. *If IH is true on the substructures of s, then it is true on s itself.*

Exercise 1.5 [⋆⋆] Prove that if $e \in LcExp$, then there are the same number of left and right parentheses in e.

1.2 Deriving Recursive Programs

We have used the method of inductive definition to characterize complicated sets. We have seen that we can analyze an element of an inductively defined set to see how it is built from smaller elements of the set. We have used this idea to write a procedure `in-S?` to decide whether a natural number is in the set S. We now use the same idea to define more general procedures that compute on inductively defined sets.

Recursive procedures rely on an important principle:

The Smaller-Subproblem Principle

If we can reduce a problem to a smaller subproblem, we can call the procedure that solves the problem to solve the subproblem.

The solution returned for the subproblem may then be used to solve the original problem. This works because each time we call the procedure, it is called with a smaller problem, until eventually it is called with a problem that can be solved directly, without another call to itself.

We illustrate this idea with a sequence of examples.

1.2.1 `list-length`

The standard Scheme procedure `length` determines the number of elements in a list.

```
> (length '(a b c))
3
> (length '((x) ()))
2
```

Let us write our own procedure, called `list-length`, that does the same thing.

We begin by writing down the *contract* for the procedure. The contract specifies the sets of possible arguments and possible return values for the procedure. The contract also may include the intended usage or behavior of the procedure. This helps us keep track of our intentions both as we write and afterwards. In code, this would be a comment; we typeset it for readability.

list-length : *List* → *Int*
usage: (list-length *l*) = the length of *l*

```
(define list-length
  (lambda (lst)
    ...))
```

We can define the set of lists by

List ::= `()` | (*Scheme value* `.` *List*)

Therefore we consider each possibility for a list. If the list is empty, then its length is 0.

list-length : *List* → *Int*
usage: (list-length *l*) = the length of *l*

```
(define list-length
  (lambda (lst)
    (if (null? lst)
        0
        ...)))
```

If a list is non-empty, then its length is one more than the length of its cdr. This gives us a complete definition.

list-length : *List* → *Int*
usage: `(list-length` *l*`) = the length of` *l*

```
(define list-length
  (lambda (lst)
    (if (null? lst)
      0
      (+ 1 (list-length (cdr lst))))))
```

We can watch `list-length` compute by using its definition.

```
(list-length '(a (b c) d))
= (+ 1 (list-length '((b c) d)))
= (+ 1 (+ 1 (list-length '(d))))
= (+ 1 (+ 1 (+ 1 (list-length '()))))
= (+ 1 (+ 1 (+ 1 0)))
= 3
```

1.2.2 `nth-element`

The standard Scheme procedure `list-ref` takes a list `lst` and a zero-based index `n` and returns element number `n` of `lst`.

```
> (list-ref '(a b c) 1)
b
```

Let us write our own procedure, called `nth-element`, that does the same thing.

Again we use the definition of *List* above.

What should `(nth-element` *lst n*`)` return when *lst* is empty? In this case, `(nth-element` *lst n*`)` is asking for an element of an empty list, so we report an error.

What should `(nth-element` *lst n*`)` return when *lst* is non-empty? The answer depends on n. If $n = 0$, the answer is simply the car of *lst*.

What should `(nth-element` *lst n*`)` return when *lst* is non-empty and $n \neq 0$? In this case, the answer is the $(n-1)$-st element of the cdr of *lst*. Since $n \in N$ and $n \neq 0$, we know that $n - 1$ must also be in N, so we can find the $(n-1)$-st element by recursively calling `nth-element`.

This leads us to the definition

```
nth-element : List × Int → SchemeVal
usage: (nth-element lst n) = the n-th element of lst
(define nth-element
  (lambda (lst n)
    (if (null? lst)
      (report-list-too-short n)
      (if (zero? n)
        (car lst)
        (nth-element (cdr lst) (- n 1))))))

(define report-list-too-short
  (lambda (n)
    (eopl:error 'nth-element
      "List too short by ~s elements.~%" (+ n 1))))
```

Here the notation **nth-element** : $List \times Int \to SchemeVal$ means that **nth-element** is a procedure that takes two arguments, a list and an integer, and returns a Scheme value. This is the same notation that is used in mathematics when we write $f : A \times B \to C$.

The procedure `report-list-too-short` reports an error condition by calling `eopl:error`. The procedure `eopl:error` aborts the computation. Its first argument is a symbol that allows the error message to identify the procedure that called `eopl:error`. The second argument is a string that is then printed in the error message. There must then be an additional argument for each instance of the character sequence `~s` in the string. The values of these arguments are printed in place of the corresponding `~s` when the string is printed. A `~%` is treated as a new line. After the error message is printed, the computation is aborted. This procedure `eopl:error` is not part of standard Scheme, but most implementations of Scheme provide such a facility. We use procedures named `report-` to report errors in a similar fashion throughout the book.

Watch how `nth-element` computes its answer:

```
  (nth-element '(a b c d e) 3)
= (nth-element   '(b c d e) 2)
= (nth-element     '(c d e) 1)
= (nth-element       '(d e) 0)
= d
```

Here `nth-element` recurs on shorter and shorter lists, and on smaller and smaller numbers.

If error checking were omitted, we would have to rely on `car` and `cdr` to complain about being passed the empty list, but their error messages would be less helpful. For example, if we received an error message from `car`, we might have to look for uses of `car` throughout our program.

Exercise 1.6 [⋆] If we reversed the order of the tests in `nth-element`, what would go wrong?

Exercise 1.7 [⋆⋆] The error message from `nth-element` is uninformative. Rewrite `nth-element` so that it produces a more informative error message, such as "`(a b c)` does not have 8 elements."

1.2.3 `remove-first`

The procedure `remove-first` should take two arguments: a symbol, *s*, and a list of symbols, *los*. It should return a list with the same elements arranged in the same order as *los*, except that the first occurrence of the symbol *s* is removed. If there is no occurrence of *s* in *los*, then *los* is returned.

```
> (remove-first 'a '(a b c))
(b c)
> (remove-first 'b '(e f g))
(e f g)
> (remove-first 'a4 '(c1 a4 c1 a4))
(c1 c1 a4)
> (remove-first 'x '())
()
```

Before we start writing the definition of this procedure, we must complete the problem specification by defining the set *List-of-Symbol* of lists of symbols. Unlike the s-lists introduced in the last section, these lists of symbols do not contain sublists.

List-of-Symbol ::= `()` | `(`*Symbol* `.` *List-of-Symbol*`)`

A list of symbols is either the empty list or a list whose car is a symbol and whose cdr is a list of symbols.

If the list is empty, there are no occurrences of s to remove, so the answer is the empty list.

```
remove-first : Sym × Listof(Sym) → Listof(Sym)
usage: (remove-first s los) returns a list with
       the same elements arranged in the same
       order as los, except that the first
       occurrence of the symbol s is removed.
(define remove-first
  (lambda (s los)
    (if (null? los)
      '()
      ...)))
```

Here we have written the contract with *Listof*(*Sym*) instead of *List-of-Symbol*. This notation will allow us to avoid many definitions like the ones above.

If *los* is non-empty, is there some case where we can determine the answer immediately? If the first element of *los* is s, say $los = (s\ s_1\ \ldots\ s_{n-1})$, the first occurrence of s is as the first element of *los*. So the result of removing it is just $(s_1\ \ldots\ s_{n-1})$.

```
remove-first : Sym × Listof(Sym) → Listof(Sym)
(define remove-first
  (lambda (s los)
    (if (null? los)
      '()
      (if (eqv? (car los) s)
        (cdr los)
        ...))))
```

If the first element of *los* is not s, say $los = (s_0\ s_1\ \ldots\ s_{n-1})$, then we know that s_0 is not the first occurrence of s. Therefore the first element of the answer must be s_0, which is the value of the expression `(car los)`. Furthermore, the first occurrence of s in *los* must be its first occurrence in $(s_1\ \ldots\ s_{n-1})$. So the rest of the answer must be the result of removing the first occurrence of s from the cdr of *los*. Since the cdr of *los* is shorter than *los*, we may recursively call `remove-first` to remove s from the cdr of *los*. So the cdr of the answer can be obtained as the value of `(remove-first s (cdr los))`. Since we know how to find the car and cdr of the answer, we can find the whole answer by combining them with `cons`, using the expression `(cons (car los) (remove-first s (cdr los)))`. With this, the complete definition of `remove-first` becomes

remove-first : *Sym* × *Listof*(*Sym*) → *Listof*(*Sym*)

```
(define remove-first
  (lambda (s los)
    (if (null? los)
      '()
      (if (eqv? (car los) s)
        (cdr los)
        (cons (car los) (remove-first s (cdr los)))))))
```

Exercise 1.8 [⋆] In the definition of `remove-first`, if the last line were replaced by `(remove-first s (cdr los))`, what function would the resulting procedure compute? Give the contract, including the usage statement, for the revised procedure.

Exercise 1.9 [⋆⋆] Define `remove`, which is like `remove-first`, except that it removes *all* occurrences of a given symbol from a list of symbols, not just the first.

1.2.4 `occurs-free?`

The procedure `occurs-free?` should take a variable *var*, represented as a Scheme symbol, and a lambda-calculus expression *exp* as defined in definition 1.1.8, and determine whether or not *var* occurs free in *exp*. We say that a variable *occurs free* in an expression *exp* if it has some occurrence in *exp* that is not inside some `lambda` binding of the same variable. For example,

```
> (occurs-free? 'x 'x)
#t
> (occurs-free? 'x 'y)
#f
> (occurs-free? 'x '(lambda (x) (x y)))
#f
> (occurs-free? 'x '(lambda (y) (x y)))
#t
> (occurs-free? 'x '((lambda (x) x) (x y)))
#t
> (occurs-free? 'x '(lambda (y) (lambda (z) (x (y z)))))
#t
```

We can solve this problem by following the grammar for lambda-calculus expressions

LcExp ::= *Identifier*
::= `(lambda (`*Identifier*`)` *LcExp*`)`
::= `(`*LcExp* *LcExp*`)`

We can summarize these cases in the rules:

- If the expression e is a variable, then the variable x occurs free in e if and only if x is the same as e.
- If the expression e is of the form `(lambda (`y`)` e'`)`, then the variable x occurs free in e if and only if y is different from x and x occurs free in e'.
- If the expression e is of the form `(`e_1 e_2`)`, then x occurs free in e if and only if it occurs free in e_1 or e_2. Here, we use "or" to mean *inclusive or*, meaning that this includes the possibility that x occurs free in both e_1 and e_2. We will generally use "or" in this sense.

You should convince yourself that these rules capture the notion of occurring "not inside a lambda-binding of x."

Exercise 1.10 [⋆] We typically use "or" to mean "inclusive or." What other meanings can "or" have?

Then it is easy to define `occurs-free?`. Since there are three alternatives to be checked, we use a Scheme `cond` rather than an `if`. In Scheme, `(or` exp_1 exp_2`)` returns a true value if either exp_1 or exp_2 returns a true value.

occurs-free? : $Sym \times LcExp \rightarrow Bool$
usage: `returns #t if the symbol` *var* `occurs free in` *exp*`, otherwise returns #f.`

```
(define occurs-free?
  (lambda (var exp)
    (cond
      ((symbol? exp) (eqv? var exp))
      ((eqv? (car exp) 'lambda)
       (and
         (not (eqv? var (car (cadr exp))))
         (occurs-free? var (caddr exp))))
      (else
        (or
          (occurs-free? var (car exp))
          (occurs-free? var (cadr exp)))))))
```

This procedure is not as readable as it might be. It is hard to tell, for example, that `(car (cadr exp))` refers to the declaration of a variable in a `lambda` expression, or that `(caddr exp)` refers to its body. We show how to improve this situation considerably in section 2.5.

1.2.5 `subst`

The procedure `subst` should take three arguments: two symbols, `new` and `old`, and an s-list, `slist`. All elements of `slist` are examined, and a new list is returned that is similar to `slist` but with all occurrences of `old` replaced by instances of `new`.

```
> (subst 'a 'b '((b c) (b () d)))
((a c) (a () d))
```

Since `subst` is defined over s-lists, its organization should reflect the definition of s-lists (definition 1.1.6)

$$\begin{aligned} &\textit{S-list} ::= (\{\textit{S-exp}\}^*) \\ &\textit{S-exp} ::= \textit{Symbol} \mid \textit{S-list} \end{aligned}$$

The Kleene star gives a concise description of the set of s-lists, but it is not so helpful for writing programs. Therefore our first step is to rewrite the grammar to eliminate the use of the Kleene star. The resulting grammar suggests that our procedure should recur on the car and cdr of an s-list.

$$\begin{aligned} \textit{S-list} &::= () \\ &::= (\textit{S-exp} \ . \ \textit{S-list}) \\ \textit{S-exp} &::= \textit{Symbol} \mid \textit{S-list} \end{aligned}$$

This example is more complex than our previous ones because the grammar for its input contains two nonterminals, *S-list* and *S-exp*. Therefore we will have two procedures, one for dealing with *S-list* and one for dealing with *S-exp*:

subst : $\textit{Sym} \times \textit{Sym} \times \textit{S-list} \to \textit{S-list}$

```
(define subst
  (lambda (new old slist)
    ...))
```

subst-in-s-exp : $\textit{Sym} \times \textit{Sym} \times \textit{S-exp} \to \textit{S-exp}$

```
(define subst-in-s-exp
  (lambda (new old sexp)
    ...))
```

Let us first work on `subst`. If the list is empty, there are no occurrences of `old` to replace.

subst : *Sym* × *Sym* × *S-list* → *S-list*

```
(define subst
  (lambda (new old slist)
    (if (null? slist)
      '()
      ...)))
```

If `slist` is non-empty, its car is a member of *S-exp* and its cdr is another s-list. In this case, the answer should be a list whose car is the result of changing `old` to `new` in the car of `slist`, and whose cdr is the result of changing `old` to `new` in the cdr of `slist`. Since the car of `slist` is an element of *S-exp*, we solve the subproblem for the car using `subst-in-s-exp`. Since the cdr of `slist` is an element of *S-list*, we recur on the cdr using `subst`:

subst : *Sym* × *Sym* × *S-list* → *S-list*

```
(define subst
  (lambda (new old slist)
    (if (null? slist)
      '()
      (cons
        (subst-in-s-exp new old (car slist))
        (subst new old (cdr slist))))))
```

Now we can move on to `subst-in-s-exp`. From the grammar, we know that the symbol expression `sexp` is either a symbol or an s-list. If it is a symbol, we need to ask whether it is the same as the symbol `old`. If it is, the answer is `new`; if it is some other symbol, the answer is the same as `sexp`. If `sexp` is an s-list, then we can recur using `subst` to find the answer.

subst-in-s-exp : *Sym* × *Sym* × *S-exp* → *S-exp*

```
(define subst-in-s-exp
  (lambda (new old sexp)
    (if (symbol? sexp)
      (if (eqv? sexp old) new sexp)
      (subst new old sexp))))
```

Since we have strictly followed the definition of *S-list* and *S-exp*, this recursion is guaranteed to halt. Since `subst` and `subst-in-s-exp` call each other recursively, we say they are *mutually recursive*.

The decomposition of `subst` into two procedures, one for each syntactic category, is an important technique. It allows us to think about one syntactic category at a time, which greatly simplifies our thinking about more complicated programs.

Exercise 1.11 [⋆] In the last line of `subst-in-s-exp`, the recursion is on `sexp` and not a smaller substructure. Why is the recursion guaranteed to halt?

Exercise 1.12 [⋆] Eliminate the one call to `subst-in-s-exp` in `subst` by replacing it by its definition and simplifying the resulting procedure. The result will be a version of `subst` that does not need `subst-in-s-exp`. This technique is called *inlining*, and is used by optimizing compilers.

Exercise 1.13 [⋆⋆] In our example, we began by eliminating the Kleene star in the grammar for *S-list*. Write `subst` following the original grammar by using `map`.

We've now developed a recipe for writing procedures that operate on inductively defined data sets. We summarize it as a slogan.

Follow the Grammar!

When defining a procedure that operates on inductively defined data, the structure of the program should be patterned after the structure of the data.

More precisely:

- Write one procedure for each nonterminal in the grammar. The procedure will be responsible for handling the data corresponding to that nonterminal, and nothing else.
- In each procedure, write one alternative for each production corresponding to that nonterminal. You may need additional case structure, but this will get you started. For each nonterminal that appears in the right-hand side, write a recursive call to the procedure for that nonterminal.

1.3 Auxiliary Procedures and Context Arguments

The *Follow-the-Grammar* recipe is powerful, but sometimes it is not sufficient. Consider the procedure `number-elements`. This procedure should take any list (`(`v_0 v_1 v_2 `...)`) and return the list (`((0` v_0`) (1` v_1`) (2` v_2`) ...)`).

A straightforward decomposition of the kind we've used so far does not solve this problem, because there is no obvious way to build the value of `(number-elements lst)` from the value of `(number-elements (cdr lst))` (but see exercise 1.36).

To solve this problem, we need to *generalize* the problem. We write a new procedure `number-elements-from` that takes an additional argument n

that specifies the number to start from. This procedure is easy to write, by recursion on the list.

number-elements-from : $Listof(SchemeVal) \times Int \rightarrow Listof(List(Int, SchemeVal))$

usage: (number-elements-from '(v_0 v_1 v_2 ...) n)
= ((n v_0) ($n+1$ v_1) ($n+2$ v_2) ...)

```
(define number-elements-from
  (lambda (lst n)
    (if (null? lst) '()
      (cons
        (list n (car lst))
        (number-elements-from (cdr lst) (+ n 1))))))
```

Here the contract header tells us that this procedure takes two arguments, a list (containing any Scheme values) and an integer, and returns a list of things, each of which is a list consisting of two elements: an integer and a Scheme value.

Once we have defined `number-elements-from`, it's easy to write the desired procedure.

number-elements : $List \rightarrow Listof(List(Int, SchemeVal))$

```
(define number-elements
  (lambda (lst)
    (number-elements-from lst 0)))
```

There are two important observations to be made here. First, the procedure `number-elements-from` has a specification that is *independent* of the specification of `number-elements`. It's very common for a programmer to write a procedure that simply calls some auxiliary procedure with some additional constant arguments. Unless we can understand what that auxiliary procedure does for *every* value of its arguments, then we can't possibly understand what the calling procedure does. This gives us a slogan:

No Mysterious Auxiliaries!

When defining an auxiliary procedure, always specify what it does on all *arguments, not just the initial values.*

Second, the two arguments to `number-elements-from` play two different roles. The first argument is the list we are working on. It gets smaller at every recursive call. The second argument, however, is an abstraction

of the *context* in which we are working. In this example, when we call `number-elements`, we end up calling `number-elements-from` on each sublist of the original list. The second argument tells us the position of the sublist in the original list. This need not decrease at a recursive call; indeed it grows, because we are passing over another element of the original list. We sometimes call this a *context argument* or *inherited attribute*.

As another example, consider the problem of summing all the values in a vector.

If we were summing the values in a list, we could follow the grammar to recur on the cdr of the list. This would get us a procedure like

list-sum : *Listof*(*Int*) $\rightarrow$ *Int*

```
(define list-sum
  (lambda (loi)
    (if (null? loi)
      0
      (+ (car loi)
         (list-sum (cdr loi))))))
```

But it is not possible to proceed in this way with vectors, because they do not decompose as readily.

Since we cannot decompose vectors, we generalize the problem to compute the sum of part of the vector. The specification of our problem is to compute

$$\sum_{i=0}^{i=length(v)-1} v_i$$

where v is a vector of integers. We generalize it by turning the upper bound into a parameter n, so that the new task is to compute

$$\sum_{i=0}^{i=n} v_i$$

where $0 \leq n < length(v)$.

This procedure is straightforward to write from its specification, using induction on its second argument n.

partial-vector-sum : *Vectorof*(*Int*) × *Int* → *Int*
usage: if $0 \leq n < length(v)$, then
(partial-vector-sum v n) = $\sum_{i=0}^{i=n} v_i$

```
(define partial-vector-sum
  (lambda (v n)
    (if (zero? n)
      (vector-ref v 0)
      (+ (vector-ref v n)
         (partial-vector-sum v (- n 1))))))
```

Since n decreases steadily to zero, a proof of correctness for this program would proceed by induction on n. Because $0 \leq n$ and $n \neq 0$, we can deduce that $0 \leq (n-1)$, so that the recursive call to the procedure `partial-vector-sum` satisfies its contract.

It is now a simple matter to solve our original problem. The procedure `partial-vector-sum` doesn't apply if the vector is of length 0, so we need to handle that case separately.

vector-sum : *Vectorof*(*Int*) → *Int*
usage: (vector-sum v) = $\sum_{i=0}^{i=length(v)-1} v_i$

```
(define vector-sum
  (lambda (v)
    (let ((n (vector-length v)))
      (if (zero? n)
        0
        (partial-vector-sum v (- n 1))))))
```

There are many other situations in which it may be helpful or necessary to introduce auxiliary variables or procedures to solve a problem. Always feel free to do so, provided that you can give an independent specification of what the new procedure is intended to do.

Exercise 1.14 [⋆⋆] Given the assumption $0 \leq n < length(v)$, prove that `partial-vector-sum` is correct.

1.4 Exercises

Getting the knack of writing recursive programs involves practice. Thus we conclude this chapter with a sequence of exercises.

In each of these exercises, assume that `s` is a symbol, `n` is a nonnegative integer, `lst` is a list, `loi` is a list of integers, `los` is a list of symbols, `slist`

is an s-list, and `x` is any Scheme value; and similarly `s1` is a symbol, `los2` is a list of symbols, `x1` is a Scheme value, etc. Also assume that `pred` is a predicate, that is, a procedure that takes any Scheme value and always returns either `#t` or `#f`. Make no other assumptions about the data unless further restrictions are given as part of a particular problem. For these exercises, there is no need to check that the input matches the description; for each procedure, assume that its input values are members of the specified sets.

Define, test, and debug each procedure. Your definition should include a contract and usage comment in the style we have used in this chapter. Feel free to define auxiliary procedures, but each auxiliary procedure you define should have its own specification, as in section 1.3.

To test these procedures, first try all the given examples. Then use other examples to test these procedures, since the given examples are not adequate to reveal all possible errors.

Exercise 1.15 [⋆] `(duple n x)` returns a list containing `n` copies of `x`.

```
> (duple 2 3)
(3 3)
> (duple 4 '(ha ha))
((ha ha) (ha ha) (ha ha) (ha ha))
> (duple 0 '(blah))
()
```

Exercise 1.16 [⋆] `(invert lst)`, where `lst` is a list of 2-lists (lists of length two), returns a list with each 2-list reversed.

```
> (invert '((a 1) (a 2) (1 b) (2 b)))
((1 a) (2 a) (b 1) (b 2))
```

Exercise 1.17 [⋆] `(down lst)` wraps parentheses around each top-level element of `lst`.

```
> (down '(1 2 3))
((1) (2) (3))
> (down '((a) (fine) (idea)))
(((a)) ((fine)) ((idea)))
> (down '(a (more (complicated)) object))
((a) ((more (complicated))) (object))
```

Exercise 1.18 [⋆] `(swapper s1 s2 slist)` returns a list the same as `slist`, but with all occurrences of `s1` replaced by `s2` and all occurrences of `s2` replaced by `s1`.

```
> (swapper 'a 'd '(a b c d))
(d b c a)
> (swapper 'a 'd '(a d () c d))
(d a () c a)
> (swapper 'x 'y '((x) y (z (x))))
((y) x (z (y)))
```

Exercise 1.19 [⋆⋆] `(list-set lst n x)` returns a list like `lst`, except that the `n`-th element, using zero-based indexing, is `x`.

```
> (list-set '(a b c d) 2 '(1 2))
(a b (1 2) d)
> (list-ref (list-set '(a b c d) 3 '(1 5 10)) 3)
(1 5 10)
```

Exercise 1.20 [⋆] `(count-occurrences s slist)` returns the number of occurrences of `s` in `slist`.

```
> (count-occurrences 'x '((f x) y (((x z) x))))
3
> (count-occurrences 'x '((f x) y (((x z) () x))))
3
> (count-occurrences 'w '((f x) y (((x z) x))))
0
```

Exercise 1.21 [⋆⋆] `(product sos1 sos2)`, where `sos1` and `sos2` are each a list of symbols without repetitions, returns a list of 2-lists that represents the Cartesian product of `sos1` and `sos2`. The 2-lists may appear in any order.

```
> (product '(a b c) '(x y))
((a x) (a y) (b x) (b y) (c x) (c y))
```

Exercise 1.22 [⋆⋆] `(filter-in pred lst)` returns the list of those elements in `lst` that satisfy the predicate `pred`.

```
> (filter-in number? '(a 2 (1 3) b 7))
(2 7)
> (filter-in symbol? '(a (b c) 17 foo))
(a foo)
```

Exercise 1.23 [⋆⋆] `(list-index pred lst)` returns the 0-based position of the first element of `lst` that satisfies the predicate `pred`. If no element of `lst` satisfies the predicate, then `list-index` returns `#f`.

```
> (list-index number? '(a 2 (1 3) b 7))
1
> (list-index symbol? '(a (b c) 17 foo))
0
> (list-index symbol? '(1 2 (a b) 3))
#f
```

Exercise 1.24 [⋆⋆] `(every? pred lst)` returns `#f` if any element of `lst` fails to satisfy `pred`, and returns `#t` otherwise.

```
> (every? number? '(a b c 3 e))
#f
> (every? number? '(1 2 3 5 4))
#t
```

Exercise 1.25 [⋆⋆] `(exists? pred lst)` returns `#t` if any element of `lst` satisfies `pred`, and returns `#f` otherwise.

```
> (exists? number? '(a b c 3 e))
#t
> (exists? number? '(a b c d e))
#f
```

Exercise 1.26 [⋆⋆] `(up lst)` removes a pair of parentheses from each top-level element of `lst`. If a top-level element is not a list, it is included in the result, as is. The value of `(up (down lst))` is equivalent to `lst`, but `(down (up lst))` is not necessarily `lst`. (See exercise 1.17.)

```
> (up '((1 2) (3 4)))
(1 2 3 4)
> (up '((x (y)) z))
(x (y) z)
```

Exercise 1.27 [⋆⋆] `(flatten slist)` returns a list of the symbols contained in `slist` in the order in which they occur when `slist` is printed. Intuitively, `flatten` removes all the inner parentheses from its argument.

```
> (flatten '(a b c))
(a b c)
> (flatten '((a) () (b ()) () (c)))
(a b c)
> (flatten '((a b) c (((d)) e)))
(a b c d e)
> (flatten '(a b (() (c))))
(a b c)
```

Exercise 1.28 [⋆⋆] `(merge loi1 loi2)`, where `loi1` and `loi2` are lists of integers that are sorted in ascending order, returns a sorted list of all the integers in `loi1` and `loi2`.

```
> (merge '(1 4) '(1 2 8))
(1 1 2 4 8)
> (merge '(35 62 81 90 91) '(3 83 85 90))
(3 35 62 81 83 85 90 90 91)
```

Exercise 1.29 [⋆⋆] `(sort loi)` returns a list of the elements of `loi` in ascending order.

```
> (sort '(8 2 5 2 3))
(2 2 3 5 8)
```

Exercise 1.30 [⋆⋆] `(sort/predicate pred loi)` returns a list of elements sorted by the predicate.

```
> (sort/predicate < '(8 2 5 2 3))
(2 2 3 5 8)
> (sort/predicate > '(8 2 5 2 3))
(8 5 3 2 2)
```

Exercise 1.31 [⋆] Write the following procedures for calculating on a bintree (definition 1.1.7): `leaf` and `interior-node`, which build bintrees, `leaf?`, which tests whether a bintree is a leaf, and `lson`, `rson`, and `contents-of`, which extract the components of a node. `contents-of` should work on both leaves and interior nodes.

Exercise 1.32 [⋆] Write a procedure `double-tree` that takes a bintree, as represented in definition 1.1.7, and produces another bintree like the original, but with all the integers in the leaves doubled.

Exercise 1.33 [⋆⋆] Write a procedure `mark-leaves-with-red-depth` that takes a bintree (definition 1.1.7), and produces a bintree of the same shape as the original, except that in the new tree, each leaf contains the integer of nodes between it and the root that contain the symbol `red`. For example, the expression

```
(mark-leaves-with-red-depth
  (interior-node 'red
    (interior-node 'bar
      (leaf 26)
      (leaf 12))
    (interior-node 'red
      (leaf 11)
      (interior-node 'quux
        (leaf 117)
        (leaf 14)))
```

which is written using the procedures defined in exercise 1.31, should return the bintree

```
(red
  (bar 1 1)
  (red 2 (quux 2 2)))
```

Exercise 1.34 [⋆⋆⋆] Write a procedure `path` that takes an integer `n` and a binary search tree `bst` (page 10) that contains the integer `n`, and returns a list of `left`s and `right`s showing how to find the node containing `n`. If `n` is found at the root, it returns the empty list.

```
> (path 17 '(14 (7 () (12 () ()))
                (26 (20 (17 () ())
                        ())
                    (31 () ()))))
(right left left)
```

Exercise 1.35 [⋆⋆⋆] Write a procedure `number-leaves` that takes a bintree, and produces a bintree like the original, except the contents of the leaves are numbered starting from 0. For example,

```
(number-leaves
  (interior-node 'foo
    (interior-node 'bar
      (leaf 26)
      (leaf 12))
    (interior-node 'baz
      (leaf 11)
      (interior-node 'quux
        (leaf 117)
        (leaf 14)))))
```

should return

```
(foo
  (bar 0 1)
  (baz
    2
    (quux 3 4)))
```

Exercise 1.36 [⋆⋆⋆] Write a procedure `g` such that `number-elements` from page 23 could be defined as

```
(define number-elements
  (lambda (lst)
    (if (null? lst) '()
      (g (list 0 (car lst)) (number-elements (cdr lst))))))
```

2 *Data Abstraction*

2.1 Specifying Data via Interfaces

Every time we decide to represent a certain set of quantities in a particular way, we are defining a new data type: the data type whose values are those representations and whose operations are the procedures that manipulate those entities.

The representation of these entities is often complex, so we do not want to be concerned with their details when we can avoid them. We may also decide to change the representation of the data. The most efficient representation is often a lot more difficult to implement, so we may wish to develop a simple implementation first and only change to a more efficient representation if it proves critical to the overall performance of a system. If we decide to change the representation of some data for any reason, we must be able to locate all parts of a program that are dependent on the representation. This is accomplished using the technique of *data abstraction*.

Data abstraction divides a data type into two pieces: an *interface* and an *implementation*. The interface tells us what the data of the type represents, what the operations on the data are, and what properties these operations may be relied on to have. The *implementation* provides a specific representation of the data and code for the operations that make use of that data representation.

A data type that is abstract in this way is said to be an *abstract data type*. The rest of the program, the *client* of the data type, manipulates the new data only through the operations specified in the interface. Thus if we wish to change the representation of the data, all we must do is change the implementation of the operations in the interface.

This is a familiar idea: when we write programs that manipulate files, most of the time we care only that we can invoke procedures that perform the open, close, read, and other typical operations on files. Similarly, most of the time, we don't care how integers are actually represented inside the machine. Our only concern is that we can perform the arithmetic operations reliably.

When the client manipulates the values of the data type only through the procedures in the interface, we say that the client code is *representation-independent*, because then the code does not rely on the representation of the values in the data type.

All the knowledge about how the data is represented must therefore reside in the code of the implementation. The most important part of an implementation is the specification of how the data is represented. We use the notation $\lceil v \rceil$ for "the representation of data v."

To make this clearer, let us consider a simple example: the data type of natural numbers. The data to be represented are the natural numbers. The interface is to consist of four procedures: `zero`, `is-zero?`, `successor`, and `predecessor`. Of course, not just any set of procedures will be acceptable as an implementation of this interface. A set of procedures will be acceptable as implementations of `zero`, `is-zero?`, `successor`, and `predecessor` only if they satisfy the four equations

$$\texttt{(zero)} = \lceil 0 \rceil$$

$$\texttt{(is-zero?}\ \lceil n \rceil\texttt{)} = \begin{cases} \texttt{\#t} & n = 0 \\ \texttt{\#f} & n \neq 0 \end{cases}$$

$$\texttt{(successor}\ \lceil n \rceil\texttt{)} = \lceil n+1 \rceil \quad (n \geq 0)$$

$$\texttt{(predecessor}\ \lceil n+1 \rceil\texttt{)} = \lceil n \rceil \quad (n \geq 0)$$

This specification does not dictate how these natural numbers are to be represented. It requires only that these procedures conspire to produce the specified behavior. Thus, the procedure `zero` must return the representation of 0. The procedure `successor`, given the representation of the number n, must return the representation of the number $n+1$, and so on. The specification says nothing about `(predecessor (zero))`, so under this specification any behavior would be acceptable.

We can now write client programs that manipulate natural numbers, and we are guaranteed that they will get correct answers, no matter what representation is in use. For example,

```
(define plus
  (lambda (x y)
    (if (is-zero? x)
      y
      (successor (plus (predecessor x) y)))))
```

will satisfy `(plus` $\lceil x \rceil$ $\lceil y \rceil$`)` $= \lceil x + y \rceil$, no matter what implementation of the natural numbers we use.

Most interfaces will contain some *constructors* that build elements of the data type, and some *observers* that extract information from values of the data type. Here we have three constructors, `zero`, `successor`, and `predecessor`, and one observer, `is-zero?`.

There are many possible representations of this interface. Let us consider three of them.

1. *Unary representation:* In the unary representation, the natural number n is represented by a list of n `#t`'s. Thus, 0 is represented by `()`, 1 is represented by `(#t)`, 2 is represented by `(#t #t)`, etc. We can define this representation inductively by:

   $\lceil 0 \rceil =$ `()`
   $\lceil n+1 \rceil =$ `(#t .` $\lceil n \rceil$`)`

   In this representation, we can satisfy the specification by writing

   ```
   (define zero (lambda () '()))
   (define is-zero? (lambda (n) (null? n)))
   (define successor (lambda (n) (cons #t n)))
   (define predecessor (lambda (n) (cdr n)))
   ```

2. *Scheme number representation:* In this representation, we simply use Scheme's internal representation of numbers (which might itself be quite complicated!). We let $\lceil n \rceil$ be the Scheme integer `n`, and define the four required entities by

   ```
   (define zero (lambda () 0))
   (define is-zero? (lambda (n) (zero? n)))
   (define successor (lambda (n) (+ n 1)))
   (define predecessor (lambda (n) (- n 1)))
   ```

3. *Bignum representation:* In the bignum representation, numbers are represented in base N, for some large integer N. The representation becomes a list consisting of numbers between 0 and $N-1$ (sometimes called *bigits* rather than digits). This representation makes it easy to represent integers that are much larger than can be represented in a machine word. For our purposes, it is convenient to keep the list with least-significant bigit first. We can define the representation inductively by

$$\lceil n \rceil = \begin{cases} \texttt{()} & n = 0 \\ \texttt{(}r\ \texttt{.}\ \lceil q \rceil\texttt{)} & n = qN + r,\ 0 \le r < N \end{cases}$$

So if $N = 16$, then $\lceil 33 \rceil =$ `(1 2)` and $\lceil 258 \rceil =$ `(2 0 1)`, since

$$258 = 2 \times 16^0 + 0 \times 16^1 + 1 \times 16^2$$

None of these implementations enforces data abstraction. There is nothing to prevent a client program from looking at the representation and determining whether it is a list or a Scheme integer. On the other hand, some languages provide direct support for data abstractions: they allow the programmer to create new interfaces and check that the new data is manipulated only through the procedures in the interface. If the representation of a type is hidden, so it cannot be exposed by any operation (including printing), the type is said to be *opaque.* Otherwise, it is said to be *transparent.*

Scheme does not provide a standard mechanism for creating new opaque types. Thus we settle for an intermediate level of abstraction: we define interfaces and rely on the writer of the client program to be discreet and use only the procedures in the interfaces.

In chapter 8, we discuss ways in which a language can enforce such protocols.

Exercise 2.1 [⋆] Implement the four required operations for bigits. Then use your implementation to calculate the factorial of 10. How does the execution time vary as this argument changes? How does the execution time vary as the base changes? Explain why.

Exercise 2.2 [⋆⋆] Analyze each of these proposed representations critically. To what extent do they succeed or fail in satisfying the specification of the data type?

Exercise 2.3 [⋆⋆] Define a representation of all the integers (negative and nonnegative) as diff-trees, where a diff-tree is a list defined by the grammar

Diff-tree ::= `(one)` | `(diff` *Diff-tree* *Diff-tree*`)`

The list `(one)` represents 1. If t_1 represents n_1 and t_2 represents n_2, then `(diff` t_1 t_2`)` is a representation of $n_1 - n_2$.

So both `(one)` and `(diff (one) (diff (one) (one)))` are representations of 1; `(diff (diff (one) (one)) (one))` is a representation of -1.

1. Show that every number has infinitely many representations in this system.
2. Turn this representation of the integers into an implementation by writing `zero`, `is-zero?`, `successor`, and `predecessor`, as specified on page 32, except that now the negative integers are also represented. Your procedures should take as input any of the multiple legal representations of an integer in this scheme. For example, if your `successor` procedure is given any of the infinitely many legal representations of 1, it should produce one of the legal representations of 2. It is permissible for different legal representations of 1 to yield different legal representations of 2.
3. Write a procedure `diff-tree-plus` that does addition in this representation. Your procedure should be optimized for the diff-tree representation, and should do its work in a constant amount of time (independent of the size of its inputs). In particular, it should not be recursive.

2.2 Representation Strategies for Data Types

When data abstraction is used, programs have the property of representation independence: programs are independent of the particular representation used to implement an abstract data type. It is then possible to change the representation by redefining the small number of procedures belonging to the interface. We frequently rely on this property in later chapters.

In this section we introduce some strategies for representing data types. We illustrate these choices using a data type of *environments*. An environment associates a value with each element of a finite set of variables. An environment may be used to associate variables with their values in a programming language implementation. A compiler may also use an environment to associate each variable name with information about that variable.

Variables may be represented in any way we please, so long as we can check two variables for equality. We choose to represent variables using Scheme symbols, but in a language without a symbol data type, variables could be represented by strings, by references into a hash table, or even by numbers (see section 3.6).

2.2.1 The Environment Interface

An environment is a function whose domain is a finite set of variables, and whose range is the set of all Scheme values. Since we adopt the usual mathematical convention that a finite function is a finite set of ordered pairs, then we need to represent all sets of the form $\{(var_1, val_1), \ldots, (var_n, val_n)\}$ where the var_i are distinct variables and the val_i are any Scheme values. We sometimes call the value of the variable *var* in an environment *env* its *binding* in *env*.

The interface to this data type has three procedures, specified as follows:

$$
\begin{aligned}
&\texttt{(empty-env)} && = \lceil \emptyset \rceil \\
&\texttt{(apply-env } \lceil f \rceil \ \textit{var}\texttt{)} && = f(\textit{var}) \\
&\texttt{(extend-env } \textit{var}\ \ v\ \lceil f \rceil\texttt{)} && = \lceil g \rceil, \\
& && \text{where } g(var_1) = \begin{cases} v & \text{if } var_1 = var \\ f(var_1) & \text{otherwise} \end{cases}
\end{aligned}
$$

The procedure `empty-env`, applied to no arguments, must produce a representation of the empty environment; `apply-env` applies a representation of an environment to a variable and `(extend-env` *var val env*`)` produces a new environment that behaves like *env*, except that its value at variable *var* is *val*. For example, the expression

```
> (define e
    (extend-env 'd 6
      (extend-env 'y 8
        (extend-env 'x 7
          (extend-env 'y 14
            (empty-env))))))
```

defines an environment e such that $e(\mathtt{d}) = 6$, $e(\mathtt{x}) = 7$, $e(\mathtt{y}) = 8$, and e is undefined on any other variables. This is, of course, only one of many different ways of building this environment. For instance, in the example above the binding of `y` to 14 is overridden by its later binding to 8.

As in the previous example, we can divide the procedures of the interface into constructors and observers. In this example, `empty-env` and `extend-env` are the constructors, and `apply-env` is the only observer.

Exercise 2.4 [⋆⋆] Consider the data type of *stacks* of values, with an interface consisting of the procedures `empty-stack`, `push`, `pop`, `top`, and `empty-stack?`. Write a specification for these operations in the style of the example above. Which operations are constructors and which are observers?

2.2.2 Data Structure Representation

We can obtain a representation of environments by observing that every environment can be built by starting with the empty environment and applying `extend-env` n times, for some $n \geq 0$, e.g.,

```
(extend-env var_n  val_n
   ...
   (extend-env var_1  val_1
     (empty-env))...)
```

So every environment can be built by an expression in the following grammar:

Env-exp ::= `(empty-env)`
::= `(extend-env` *Identifier Scheme-value Env-exp*`)`

We could represent environments using the same grammar to describe a set of lists. This would give the implementation shown in figure 2.1. The procedure `apply-env` looks at the data structure `env` representing an environment, determines what kind of environment it represents, and does the right thing. If it represents the empty environment, then an error is reported. If it represents an environment built by `extend-env`, then it checks to see if the variable it is looking for is the same as the one bound in the environment. If it is, then the saved value is returned. Otherwise, the variable is looked up in the saved environment.

This is a very common pattern of code. We call it the *interpreter recipe*:

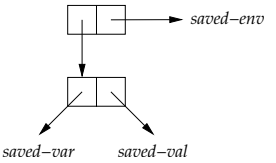
The Interpreter Recipe

1. *Look at a piece of data.*
2. *Decide what kind of data it represents.*
3. *Extract the components of the datum and do the right thing with them.*

```
Env = (empty-env) | (extend-env Var SchemeVal Env)
Var = Sym

empty-env : () → Env
(define empty-env
  (lambda () (list 'empty-env)))

extend-env : Var × SchemeVal × Env → Env
(define extend-env
  (lambda (var val env)
    (list 'extend-env var val env)))

apply-env : Env × Var → SchemeVal
(define apply-env
  (lambda (env search-var)
    (cond
      ((eqv? (car env) 'empty-env)
       (report-no-binding-found search-var))
      ((eqv? (car env) 'extend-env)
       (let ((saved-var (cadr env))
             (saved-val (caddr env))
             (saved-env (cadddr env)))
         (if (eqv? search-var saved-var)
           saved-val
           (apply-env saved-env search-var))))
      (else
        (report-invalid-env env)))))

(define report-no-binding-found
  (lambda (search-var)
    (eopl:error 'apply-env "No binding for ~s" search-var)))

(define report-invalid-env
  (lambda (env)
    (eopl:error 'apply-env "Bad environment: ~s" env)))
```

Figure 2.1 A data-structure representation of environments

Exercise 2.5 [⋆] We can use any data structure for representing environments, if we can distinguish empty environments from non-empty ones, and in which one can extract the pieces of a non-empty environment. Implement environments using a representation in which the empty environment is represented as the empty list, and in which `extend-env` builds an environment that looks like



This is called an *a-list* or *association-list* representation.

Exercise 2.6 [⋆] Invent at least three different representations of the environment interface and implement them.

Exercise 2.7 [⋆] Rewrite `apply-env` in figure 2.1 to give a more informative error message.

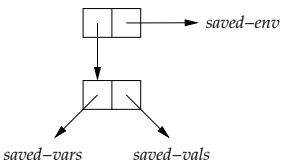
Exercise 2.8 [⋆] Add to the environment interface an observer called `empty-env?` and implement it using the a-list representation.

Exercise 2.9 [⋆] Add to the environment interface an observer called `has-binding?` that takes an environment *env* and a variable s and tests to see if s has an associated value in *env*. Implement it using the a-list representation.

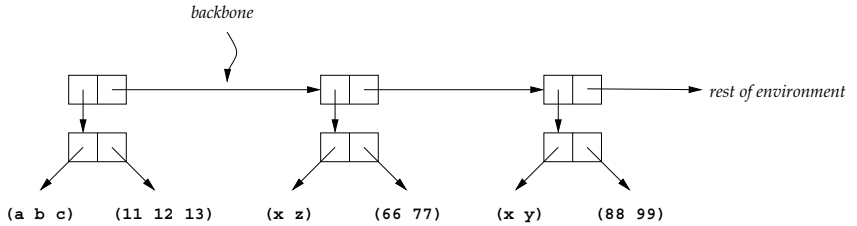
Exercise 2.10 [⋆] Add to the environment interface a constructor `extend-env*`, and implement it using the a-list representation. This constructor takes a list of variables, a list of values of the same length, and an environment, and is specified by

$$\texttt{(extend-env*}\ \texttt{(}var_1\ \ldots\ var_k\texttt{)}\ \texttt{(}val_1\ \ldots\ val_k\texttt{)}\ \lceil f \rceil\texttt{)} = \lceil g \rceil,$$
$$\text{where } g(var) = \begin{cases} val_i & \text{if } var = var_i \text{ for some } i \text{ such that } 1 \le i \le k \\ f(var) & \text{otherwise} \end{cases}$$

Exercise 2.11 [⋆⋆] A naive implementation of `extend-env*` from the preceding exercise requires time proportional to k to run. It is possible to represent environments so that `extend-env*` requires only constant time: represent the empty environment by the empty list, and represent a non-empty environment by the data structure



Such an environment might look like



This is called the *ribcage* representation. The environment is represented as a list of pairs called *ribs*; each left rib is a list of variables and each right rib is the corresponding list of values.

Implement the environment interface, including `extend-env*`, in this representation.

2.2.3 Procedural Representation

The environment interface has an important property: it has exactly one observer, `apply-env`. This allows us to represent an environment as a Scheme procedure that takes a variable and returns its associated value.

To do this, we define `empty-env` and `extend-env` to return procedures that, when applied, do the same thing that `apply-env` did in the preceding section. This gives us the following implementation.

Env = *Var* → *SchemeVal*

empty-env : () → *Env*
```
(define empty-env
  (lambda ()
    (lambda (search-var)
      (report-no-binding-found search-var))))
```

extend-env : *Var* × *SchemeVal* × *Env* → *Env*
```
(define extend-env
  (lambda (saved-var saved-val saved-env)
    (lambda (search-var)
      (if (eqv? search-var saved-var)
        saved-val
        (apply-env saved-env search-var)))))
```

apply-env : *Env* × *Var* → *SchemeVal*
```
(define apply-env
  (lambda (env search-var)
    (env search-var)))
```

If the empty environment, created by invoking `empty-env`, is passed any variable whatsoever, it indicates with an error message that the given variable is not in its domain. The procedure `extend-env` returns a new procedure that represents the extended environment. This procedure, when passed a variable `search-var`, checks to see if the variable it is looking for is the same as the one bound in the environment. If it is, then the saved value is returned. Otherwise, the variable is looked up in the saved environment.

We call this a *procedural representation*, in which the data is represented by its *action under* `apply-env`.

The case of a data type with a single observer is less rare than one might think. For example, if the data being represented is a set of functions, then it can be represented by its action under application. In this case, we can extract the interface and the procedural representation by the following recipe:

1. Identify the lambda expressions in the client code whose evaluation yields values of the type. Create a constructor procedure for each such lambda expression. The parameters of the constructor procedure will be the free variables of the lambda expression. Replace each such lambda expression in the client code by an invocation of the corresponding constructor.

2. Define an `apply-` procedure like `apply-env` above. Identify all the places in the client code, including the bodies of the constructor procedures, where a value of the type is applied. Replace each such application by an invocation of the `apply-` procedure.

If these steps are carried out, the interface will consist of all the constructor procedures and the `apply-` procedure, and the client code will be representation-independent: it will not rely on the representation, and we will be free to substitute another implementation of the interface, such as the one we describe in section 2.2.2.

If the implementation language does not allow higher-order procedures, then one can perform the additional step of implementing the resulting interface using a data structure representation and the interpreter recipe, as in the preceding section. This process is called *defunctionalization*. The derivation of the data structure representation of environments is a simple example of defunctionalization. The relation between procedural and defunctionalized representations will be a recurring theme in this book.

Exercise 2.12 [⋆] Implement the stack data type of exercise 2.4 using a procedural representation.

Exercise 2.13 [⋆⋆] Extend the procedural representation to implement `empty-env?` by representing the environment by a list of two procedures: one that returns the value associated with a variable, as before, and one that returns whether or not the environment is empty.

Exercise 2.14 [⋆⋆] Extend the representation of the preceding exercise to include a third procedure that implements `has-binding?` (see exercise 2.9).

2.3 Interfaces for Recursive Data Types

We spent much of chapter 1 manipulating recursive data types. For example, we defined lambda-calculus expressions in definition 1.1.8 by the grammar

$$\begin{aligned} \textit{Lc-exp} &::= \textit{Identifier} \\ &::= \texttt{(lambda (}\textit{Identifier}\texttt{)}\ \textit{Lc-exp}\texttt{)} \\ &::= \texttt{(}\textit{Lc-exp}\ \ \textit{Lc-exp}\texttt{)} \end{aligned}$$

and we wrote procedures like `occurs-free?`. As we mentioned at the time, the definition of `occurs-free?` in section 1.2.4 is not as readable as it might be. It is hard to tell, for example, that `(car (cadr exp))` refers to the declaration of a variable in a `lambda` expression, or that `(caddr exp)` refers to its body.

We can improve this situation by introducing an interface for lambda-calculus expressions. Our interface will have constructors and two kinds of observers: predicates and extractors.

The constructors are:

var-exp	: $\textit{Var} \to \textit{Lc-exp}$
lambda-exp	: $\textit{Var} \times \textit{Lc-exp} \to \textit{Lc-exp}$
app-exp	: $\textit{Lc-exp} \times \textit{Lc-exp} \to \textit{Lc-exp}$

The predicates are:

var-exp?	: $\textit{Lc-exp} \to \textit{Bool}$
lambda-exp?	: $\textit{Lc-exp} \to \textit{Bool}$
app-exp?	: $\textit{Lc-exp} \to \textit{Bool}$

Finally, the extractors are

var-exp->var	: *Lc-exp* → *Var*
lambda-exp->bound-var	: *Lc-exp* → *Var*
lambda-exp->body	: *Lc-exp* → *Lc-exp*
app-exp->rator	: *Lc-exp* → *Lc-exp*
app-exp->rand	: *Lc-exp* → *Lc-exp*

Each of these extracts the corresponding portion of the lambda-calculus expression. We can now write a version of `occurs-free?` that depends only on the interface.

occurs-free? : *Sym* × *LcExp* → *Bool*

```
(define occurs-free?
  (lambda (search-var exp)
    (cond
      ((var-exp? exp) (eqv? search-var (var-exp->var exp)))
      ((lambda-exp? exp)
       (and
         (not (eqv? search-var (lambda-exp->bound-var exp)))
         (occurs-free? search-var (lambda-exp->body exp))))
      (else
        (or
          (occurs-free? search-var (app-exp->rator exp))
          (occurs-free? search-var (app-exp->rand exp)))))))
```

This works on any representation of lambda-calculus expressions, so long as they are built using these constructors.

We can write down a general recipe for designing an interface for a recursive data type:

Designing an interface for a recursive data type

1. *Include one constructor for each kind of data in the data type.*
2. *Include one predicate for each kind of data in the data type.*
3. *Include one extractor for each piece of data passed to a constructor of the data type.*

Exercise 2.15 [⋆] Implement the lambda-calculus expression interface for the representation specified by the grammar above.

Exercise 2.16 [⋆] Modify the implementation to use a representation in which there are no parentheses around the bound variable in a `lambda` expression.

Exercise 2.17 [⋆] Invent at least two other representations of the data type of lambda-calculus expressions and implement them.

Exercise 2.18 [⋆] We usually represent a sequence of values as a list. In this representation, it is easy to move from one element in a sequence to the next, but it is hard to move from one element to the preceding one without the help of context arguments. Implement non-empty bidirectional sequences of integers, as suggested by the grammar

$$\textit{NodeInSequence} ::= (\textit{Int}\ \ \textit{Listof}(\textit{Int})\ \ \textit{Listof}(\textit{Int}))$$

The first list of numbers is the elements of the sequence preceding the current one, in reverse order, and the second list is the elements of the sequence after the current one. For example, `(6 (5 4 3 2 1) (7 8 9))` represents the list `(1 2 3 4 5 6 7 8 9)`, with the focus on the element `6`.

In this representation, implement the procedure `number->sequence`, which takes a number and produces a sequence consisting of exactly that number. Also implement `current-element`, `move-to-left`, `move-to-right`, `insert-to-left`, `insert-to-right`, `at-left-end?`, and `at-right-end?`.

For example:

```
> (number->sequence 7)
(7 () ())
> (current-element '(6 (5 4 3 2 1) (7 8 9)))
6
> (move-to-left '(6 (5 4 3 2 1) (7 8 9)))
(5 (4 3 2 1) (6 7 8 9))
> (move-to-right '(6 (5 4 3 2 1) (7 8 9)))
(7 (6 5 4 3 2 1) (8 9))
> (insert-to-left 13 '(6 (5 4 3 2 1) (7 8 9)))
(6 (13 5 4 3 2 1) (7 8 9))
> (insert-to-right 13 '(6 (5 4 3 2 1) (7 8 9)))
(6 (5 4 3 2 1) (13 7 8 9))
```

The procedure `move-to-right` should fail if its argument is at the right end of the sequence, and the procedure `move-to-left` should fail if its argument is at the left end of the sequence.

Exercise 2.19 [⋆] A binary tree with empty leaves and with interior nodes labeled with integers could be represented using the grammar

$$\textit{Bintree} ::= () \mid (\textit{Int}\ \ \textit{Bintree}\ \ \textit{Bintree})$$

In this representation, implement the procedure `number->bintree`, which takes a number and produces a binary tree consisting of a single node containing that number. Also implement `current-element`, `move-to-left-son`, `move-to-right-son`, `at-leaf?`, `insert-to-left`, and `insert-to-right`. For example,

```
> (number->bintree 13)
(13 () ())
> (define t1 (insert-to-right 14
               (insert-to-left 12
                 (number->bintree 13)))
> t1
(13
  (12 () ())
  (14 () ()))
> (move-to-left t1)
(12 () ())
> (current-element (move-to-left t1))
12
> (at-leaf? (move-to-right (move-to-left t1)))
#t
> (insert-to-left 15 t1)
(13
  (15
    (12 () ())
    ())
  (14 () ()))
```

Exercise 2.20 [⋆⋆⋆] In the representation of binary trees in exercise 2.19 it is easy to move from a parent node to one of its sons, but it is impossible to move from a son to its parent without the help of context arguments. Extend the representation of lists in exercise 2.18 to represent nodes in a binary tree. As a hint, consider representing the portion of the tree above the current node by a reversed list, as in exercise 2.18.

In this representation, implement the procedures from exercise 2.19. Also implement `move-up`, `at-root?`, and `at-leaf?`.

2.4 A Tool for Defining Recursive Data Types

For complicated data types, applying the recipe for constructing an interface can quickly become tedious. In this section, we introduce a tool for automatically constructing and implementing such interfaces in Scheme. The interfaces constructed by this tool will be similar, but not identical, to the interface constructed in the preceding section.

Consider again the data type of lambda-calculus expressions, as discussed in the preceding section. We can implement an interface for lambda-calculus expressions by writing

```
(define-datatype lc-exp lc-exp?
  (var-exp
    (var identifier?))
  (lambda-exp
    (bound-var identifier?)
    (body lc-exp?))
  (app-exp
    (rator lc-exp?)
    (rand lc-exp?)))
```

Here the names `var-exp`, `var`, `bound-var`, `app-exp`, `rator`, and `rand` abbreviate *variable expression*, *variable*, *bound variable*, *application expression*, *operator*, and *operand*, respectively.

This expression declares three constructors, `var-exp`, `lambda-exp`, and `app-exp`, and a single predicate `lc-exp?`. The three constructors check their arguments with the predicates `identifier?` and `lc-exp?` to make sure that the arguments are valid, so if an lc-exp is constructed using only these constructors, we can be certain that it and all its subexpressions are legal lc-exps. This allows us to ignore many checks while processing lambda expressions.

In place of the various predicates and extractors, we use the form `cases` to determine the variant to which an object of a data type belongs, and to extract its components. To illustrate this form, we can rewrite `occurs-free?` (page 43) using the data type `lc-exp`:

occurs-free? : $Sym \times LcExp \rightarrow Bool$

```
(define occurs-free?
  (lambda (search-var exp)
    (cases lc-exp exp
      (var-exp (var) (eqv? var search-var))
      (lambda-exp (bound-var body)
        (and
          (not (eqv? search-var bound-var))
          (occurs-free? search-var body)))
      (app-exp (rator rand)
        (or
          (occurs-free? search-var rator)
          (occurs-free? search-var rand))))))
```

To see how this works, assume that `exp` is a lambda-calculus expression that was built by `app-exp`. For this value of `exp`, the `app-exp` case would be selected, `rator` and `rand` would be bound to the two subexpressions, and the expression

```
(or
  (occurs-free? search-var rator)
  (occurs-free? search-var rand))
```

would be evaluated, just as if we had written

```
(if (app-exp? exp)
  (let ((rator (app-exp->rator exp))
        (rand (app-exp->rand exp)))
    (or
      (occurs-free? search-var rator)
      (occurs-free? search-var rand)))
...)
```

The recursive calls to `occurs-free?` work similarly to finish the calculation.

In general, a `define-datatype` declaration has the form

`(define-datatype` *type-name type-predicate-name*
  {`(`*variant-name* {`(`*field-name predicate*`)`}*`)`}$^+$`)`

This creates a data type, named *type-name*, with some *variants*. Each variant has a variant-name and zero or more fields, each with its own field-name and associated predicate. No two types may have the same name and no two variants, even those belonging to different types, may have the same name. Also, type names cannot be used as variant names. Each field predicate must be a Scheme predicate.

For each variant, a new constructor procedure is created that is used to create data values belonging to that variant. These procedures are named after their variants. If there are n fields in a variant, its constructor takes n arguments, tests each of them with its associated predicate, and returns a new value of the given variant with the i-th field containing the i-th argument value.

The *type-predicate-name* is bound to a predicate. This predicate determines if its argument is a value belonging to the named type.

A record can be defined as a data type with a single variant. To distinguish data types with only one variant, we use a naming convention. When there is a single variant, we name the constructor `a-`*type-name* or `an-`*type-name*; otherwise, the constructors have names like *variant-name-type-name*.

Data types built by `define-datatype` may be mutually recursive. For example, consider the grammar for s-lists from section 1.1:

$$\textit{S-list} ::= (\{\textit{S-exp}\}^*)$$
$$\textit{S-exp} ::= \textit{Symbol} \mid \textit{S-list}$$

The data in an s-list could be represented by the data type `s-list` defined by

```
(define-datatype s-list s-list?
  (empty-s-list)
  (non-empty-s-list
    (first s-exp?)
    (rest s-list?)))

(define-datatype s-exp s-exp?
  (symbol-s-exp
    (sym symbol?))
  (s-list-s-exp
    (slst s-list?)))
```

The data type `s-list` gives its own representation of lists by using `(empty-s-list)` and `non-empty-s-list` in place of `()` and `cons`; if we wanted to specify that Scheme lists be used instead, we could have written

```
(define-datatype s-list s-list?
  (an-s-list
    (sexps (list-of s-exp?))))

(define list-of
  (lambda (pred)
    (lambda (val)
      (or (null? val)
        (and (pair? val)
          (pred (car val))
          ((list-of pred) (cdr val)))))))
```

Here (`list-of` *pred*) builds a predicate that tests to see if its argument is a list, and that each of its elements satisfies *pred*.

The general syntax of `cases` is

```
(cases type-name expression
  {(variant-name ({field-name}*) consequent)}*
  (else default))
```

The form specifies the type, the expression yielding the value to be examined, and a sequence of clauses. Each clause is labeled with the name of a variant of the given type and the names of its fields. The `else` clause is optional. First, *expression* is evaluated, resulting in some value v of *type-name*. If v is a variant of *variant-name*, then the corresponding clause is selected. Each of the *field-names* is bound to the value of the corresponding field of v. Then the *consequent* is evaluated within the scope of these bindings and its value returned. If v is not one of the variants, and an `else` clause has been specified, *default* is evaluated and its value returned. If there is no `else` clause, then there must be a clause for *every* variant of that data type.

The form `cases` binds its variables positionally: the i-th variable is bound to the value in the i-th field. So we could just as well have written

```
(app-exp (exp1 exp2)
  (or
    (occurs-free? search-var exp1)
    (occurs-free? search-var exp2)))
```

instead of

```
(app-exp (rator rand)
  (or
    (occurs-free? search-var rator)
    (occurs-free? search-var rand)))
```

The forms `define-datatype` and `cases` provide a convenient way of defining an inductive data type, but it is not the only way. Depending on the application, it may be valuable to use a special-purpose representation that is more compact or efficient, taking advantage of special properties of the data. These advantages are gained at the expense of having to write the procedures in the interface by hand.

The form `define-datatype` is an example of a *domain-specific language*. A domain-specific language is a small language for describing a single task among a small, well-defined set of tasks. In this case, the task was defining a recursive data type. Such a language may lie inside a general-purpose language, as `define-datatype` does, or it may be a standalone language with

its own set of tools. In general, one constructs such a language by identifying the possible variations in the set of tasks, and then designing a language that describes those variations. This is often a very useful strategy.

Exercise 2.21 [⋆] Implement the data type of environments, as in section 2.2.2, using `define-datatype`. Then include `has-binding?` of exercise 2.9.

Exercise 2.22 [⋆] Using `define-datatype`, implement the stack data type of exercise 2.4.

Exercise 2.23 [⋆] The definition of `lc-exp` ignores the condition in definition 1.1.8 that says "*Identifier* is any symbol other than `lambda`." Modify the definition of `identifier?` to capture this condition. As a hint, remember that any predicate can be used in `define-datatype`, even ones you define.

Exercise 2.24 [⋆] Here is a definition of binary trees using `define-datatype`.

```
(define-datatype bintree bintree?
  (leaf-node
    (num integer?))
  (interior-node
    (key symbol?)
    (left bintree?)
    (right bintree?)))
```

Implement a `bintree-to-list` procedure for binary trees, so that `(bintree-to-list (interior-node 'a (leaf-node 3) (leaf-node 4)))` returns the list

```
(interior-node
  a
  (leaf-node 3)
  (leaf-node 4))
```

Exercise 2.25 [⋆⋆] Use `cases` to write `max-interior`, which takes a binary tree of integers (as in the preceding exercise) with at least one interior node and returns the symbol associated with an interior node with a maximal leaf sum.

```
> (define tree-1
    (interior-node 'foo (leaf-node 2) (leaf-node 3)))
> (define tree-2
    (interior-node 'bar (leaf-node -1) tree-1))
> (define tree-3
    (interior-node 'baz tree-2 (leaf-node 1)))
> (max-interior tree-2)
foo
> (max-interior tree-3)
baz
```

The last invocation of `max-interior` might also have returned `foo`, since both the `foo` and `baz` nodes have a leaf sum of 5.

Exercise 2.26 [⋆⋆] Here is another version of exercise 1.33. Consider a set of trees given by the following grammar:

Red-blue-tree ::= *Red-blue-subtree*
Red-blue-subtree ::= `(red-node` *Red-blue-subtree* *Red-blue-subtree*`)`
::= `(blue-node` {*Red-blue-subtree*}* `)`
::= `(leaf-node` *Int*`)`

Write an equivalent definition using `define-datatype`, and use the resulting interface to write a procedure that takes a tree and builds a tree of the same shape, except that each leaf node is replaced by a leaf node that contains the number of red nodes on the path between it and the root.

2.5 Abstract Syntax and Its Representation

A grammar usually specifies a particular representation of an inductive data type: one that uses the strings or values generated by the grammar. Such a representation is called *concrete syntax*, or *external* representation.

Consider, for example, the set of lambda-calculus expressions defined in definition 1.1.8. This gives a concrete syntax for lambda-calculus expressions. We might have used some other concrete syntax for lambda-calculus expressions. For example, we could have written

Lc-exp ::= *Identifier*
::= `proc` *Identifier* `=>` *Lc-exp*
::= *Lc-exp* `(` *Lc-exp* `)`

to define lambda-calculus expressions as a different set of strings.

In order to process such data, we need to convert it to an *internal* representation. The `define-datatype` form provides a convenient way of defining such an internal representation. We call this *abstract syntax*. In the abstract syntax, terminals such as parentheses need not be stored, because they convey no information. On the other hand, we want to make sure that the data structure allows us to determine what kind of lambda-calculus expression it represents, and to extract its components. The data type `lc-exp` on page 46 allows us to do both of these things easily.

It is convenient to visualize the internal representation as an *abstract syntax tree.* Figure 2.2 shows the abstract syntax tree of the lambda-calculus expression `(lambda (x) (f (f x)))`, using the data type `lc-exp`. Each internal node of the tree is labeled with the associated production name. Edges are labeled with the name of the corresponding nonterminal occurrence. Leaves correspond to terminal strings.

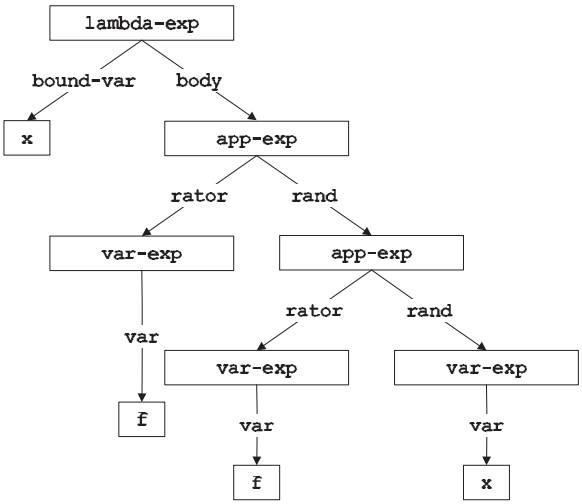


Figure 2.2 Abstract syntax tree for `(lambda (x) (f (f x)))`

To create an abstract syntax for a given concrete syntax, we must name each production of the concrete syntax and each occurrence of a nonterminal in each production. It is straightforward to generate `define-datatype` declarations for the abstract syntax. We create one `define-datatype` for each nonterminal, with one variant for each production.

We can summarize the choices we have made in figure 2.2 using the following concise notation:

Lc-exp ::= *Identifier*
`var-exp (var)`

::= `(lambda (`*Identifier*`)` *Lc-exp*`)`
`lambda-exp (bound-var body)`

::= `(`*Lc-exp* *Lc-exp*`)`
`app-exp (rator rand)`

Such notation, which specifies both concrete and abstract syntax, is used throughout this book.

Having made the distinction between concrete syntax, which is primarily useful for humans, and abstract syntax, which is primarily useful for computers, we now consider how to convert from one syntax to the other.

If the concrete syntax is a set of strings of characters, it may be a complex undertaking to derive the corresponding abstract syntax tree. This task is called *parsing* and is performed by a *parser*. Because writing a parser is difficult in general, it is best performed by a tool called a *parser generator*. A parser generator takes as input a grammar and produces a parser. Since the grammars are processed by a tool, they must be written in some machine-readable language: a domain-specific language for writing grammars. There are many parser generators available.

If the concrete syntax is given as a set of lists, the parsing process is considerably simplified. For example, the grammar for lambda-calculus expressions at the beginning of this section specified a set of lists, as did the grammar for `define-datatype` on page 47. In this case, the Scheme `read` routine automatically parses strings into lists and symbols. It is then easier to parse these list structures into abstract syntax trees as in `parse-expression`.

parse-expression : *SchemeVal* → *LcExp*

```
(define parse-expression
  (lambda (datum)
    (cond
      ((symbol? datum) (var-exp datum))
      ((pair? datum)
       (if (eqv? (car datum) 'lambda)
         (lambda-exp
           (car (cadr datum))
           (parse-expression (caddr datum)))
         (app-exp
           (parse-expression (car datum))
           (parse-expression (cadr datum)))))
      (else (report-invalid-concrete-syntax datum)))))
```

It is usually straightforward to convert an abstract syntax tree back to a list-and-symbol representation. If we do this, the Scheme print routines will then display it in a list-based concrete syntax. This is performed by `unparse-lc-exp`:

unparse-lc-exp : *LcExp* → *SchemeVal*

```
(define unparse-lc-exp
  (lambda (exp)
    (cases lc-exp exp
      (var-exp (var) var)
      (lambda-exp (bound-var body)
        (list 'lambda (list bound-var)
          (unparse-lc-exp body)))
      (app-exp (rator rand)
        (list
```

```
            (unparse-lc-exp rator) (unparse-lc-exp rand))))))
```

Exercise 2.27 [⋆] Draw the abstract syntax tree for the lambda calculus expressions

```
((lambda (a) (a b)) c)

(lambda (x)
  (lambda (y)
    ((lambda (x)
       (x y))
     x)))
```

Exercise 2.28 [⋆] Write an unparser that converts the abstract syntax of an lc-exp into a string that matches the second grammar in this section (page 52).

Exercise 2.29 [⋆] Where a Kleene star or plus (page 7) is used in concrete syntax, it is most convenient to use a *list* of associated subtrees when constructing an abstract syntax tree. For example, if the grammar for lambda-calculus expressions had been

Lc-exp ::= *Identifier*
`var-exp (var)`

::= `(lambda (`{*Identifier*}*`)` *Lc-exp*`)`
`lambda-exp (bound-vars body)`

::= `(`*Lc-exp* {*Lc-exp*}*`)`
`app-exp (rator rands)`

then the predicate for the `bound-vars` field could be `(list-of identifier?)`, and the predicate for the `rands` field could be `(list-of lc-exp?)`. Write a `define-datatype` and a parser for this grammar that works in this way.

Exercise 2.30 [⋆⋆] The procedure `parse-expression` as defined above is fragile: it does not detect several possible syntactic errors, such as `(a b c)`, and aborts with inappropriate error messages for other expressions, such as `(lambda)`. Modify it so that it is robust, accepting any s-exp and issuing an appropriate error message if the s-exp does not represent a lambda-calculus expression.

Exercise 2.31 [⋆⋆] Sometimes it is useful to specify a concrete syntax as a sequence of symbols and integers, surrounded by parentheses. For example, one might define the set of *prefix lists* by

Prefix-list ::= `(`*Prefix-exp*`)`
Prefix-exp ::= *Int*
::= `-` *Prefix-exp* *Prefix-exp*

so that `(- - 3 2 - 4 - 12 7)` is a legal prefix list. This is sometimes called *Polish prefix notation*, after its inventor, Jan Łukasiewicz. Write a parser to convert a prefix-list to the abstract syntax

```
(define-datatype prefix-exp prefix-exp?
  (const-exp
    (num integer?))
  (diff-exp
    (operand1 prefix-exp?)
    (operand2 prefix-exp?)))
```

so that the example above produces the same abstract syntax tree as the sequence of constructors

```
(diff-exp
  (diff-exp
    (const-exp 3)
    (const-exp 2))
  (diff-exp
    (const-exp 4)
    (diff-exp
      (const-exp 12)
      (const-exp 7))))
```

As a hint, consider writing a procedure that takes a list and produces a `prefix-exp` and the list of leftover list elements.

3 *Expressions*

In this chapter, we study the binding and scoping of variables. We do this by presenting a sequence of small languages that illustrate these concepts. We write specifications for these languages, and implement them using interpreters, following the interpreter recipe from chapter 1. Our specifications and interpreters take a context argument, called the *environment*, which keeps track of the meaning of each variable in the expression being evaluated.

3.1 Specification and Implementation Strategy

Our specification will consist of assertions of the form

$$(\texttt{value-of}\ \ exp\ \ \rho) = val$$

meaning that the value of expression exp in environment ρ should be val. We write down rules of inference and equations, like those in chapter 1, that will enable us to derive such assertions. We use the rules and equations by hand to find the intended value of some expressions.

But our goal is to write a program that implements our language. The overall picture is shown in figure 3.1(a). We start with the text of the program written in the language we are implementing. This is called the *source language* or the *defined language*. Program text (a program in the source language) is passed through a front end that converts it to an abstract syntax tree. The syntax tree is then passed to the interpreter, which is a program that looks at a data structure and performs some actions that depend on its structure. Of course the interpreter is itself written in some language. We call that language the *implementation language* or the *defining language*. Most of our implementations will follow this pattern.

Another common organization is shown in figure 3.1(b). There the interpreter is replaced by a compiler, which translates the abstract syntax tree into a program in some other language (the *target language*), and that program is executed. That target language may be executed by an interpreter, as in figure 3.1(b), or it may be translated into some even lower-level language for execution.

Most often, the target language is a machine language, which is interpreted by a hardware machine. Yet another possibility is that the target machine is a special-purpose language that is simpler than the original and for which it is relatively simple to write an interpreter. This allows the program to be compiled once and then executed on many different hardware platforms. For historical reasons, such a target language is often called a *byte code*, and its interpreter is called a *virtual machine*.

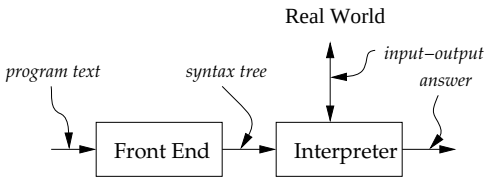
A compiler is typically divided into two parts: an *analyzer* that attempts to deduce useful information about the program, and a *translator* that does the translation, possibly using information from the analyzer. Each of these phases may be specified either by rules of inference or a special-purpose specification language, and then implemented. We study some simple analyzers and translators in chapters 6 and 7.

No matter what implementation strategy we use, we need a *front end* that converts programs into abstract syntax trees. Because programs are just strings of characters, our front end needs to group these characters into meaningful units. This grouping is usually divided into two stages: *scanning* and *parsing*.

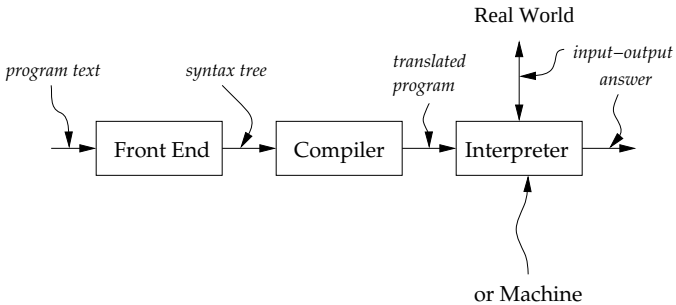
Scanning is the process of dividing the sequence of characters into words, numbers, punctuation, comments, and the like. These units are called *lexical items*, *lexemes*, or most often *tokens*. We refer to the way in which a program should be divided up into tokens as the *lexical specification* of the language. The scanner takes a sequence of characters and produces a sequence of tokens.

Parsing is the process of organizing the sequence of tokens into hierarchical syntactic structures such as expressions, statements, and blocks. This is like organizing (diagramming) a sentence into clauses. We refer to this as the *syntactic* or *grammatical* structure of the language. The parser takes a sequence of tokens from the scanner and produces an abstract syntax tree.

The standard approach to building a front end is to use a *parser generator*. A parser generator is a program that takes as input a lexical specification and a grammar, and produces as output a scanner and parser for them.



(a) Execution via interpreter



(b) Execution via Compiler

Figure 3.1 Block diagrams for a language-processing system

Parser generator systems are available for most major languages. If no parser generator is available, or none is suitable for the application, one can choose to build a scanner and parser by hand. This process is described in compiler textbooks. The parsing technology and associated grammars we use are designed for simplicity in the context of our very specialized needs.

Another approach is to ignore the details of the concrete syntax and to write our expressions as list structures, as we did for lambda-calculus expressions with the procedure `parse-expression` in section 2.5 and exercise 2.31.

Program ::= *Expression*
`a-program (exp1)`

Expression ::= *Number*
`const-exp (num)`

Expression ::= `-(`*Expression* `,` *Expression*`)`
`diff-exp (exp1 exp2)`

Expression ::= `zero? (`*Expression*`)`
`zero?-exp (exp1)`

Expression ::= `if` *Expression* `then` *Expression* `else` *Expression*
`if-exp (exp1 exp2 exp3)`

Expression ::= *Identifier*
`var-exp (var)`

Expression ::= `let` *Identifier* `=` *Expression* `in` *Expression*
`let-exp (var exp1 body)`

Figure 3.2 Syntax for the LET language

3.2 LET: A Simple Language

We begin by specifying a very simple language, which we call LET, after its most interesting feature.

3.2.1 Specifying the Syntax

Figure 3.2 shows the syntax of our simple language. In this language, a program is just an expression. An expression is either an integer constant, a difference expression, a zero-test expression, a conditional expression, a variable, or a `let` expression.

Here is a simple expression in this language and its representation as abstract syntax.

```
(scan&parse "-(55, -(x,11))")
#(struct:a-program
   #(struct:diff-exp
      #(struct:const-exp 55)
      #(struct:diff-exp
         #(struct:var-exp x)
         #(struct:const-exp 11))))
```

3.2.2 Specification of Values

An important part of the specification of any programming language is the set of values that the language manipulates. Each language has at least two such sets: the *expressed values* and the *denoted values*. The expressed values are the possible values of expressions, and the denoted values are the values bound to variables.

In the languages of this chapter, the expressed and denoted values will always be the same. They will start out as

$$\begin{aligned} ExpVal &= Int + Bool \\ DenVal &= Int + Bool \end{aligned}$$

Chapter 4 presents languages in which expressed and denoted values are different.

In order to make use of this definition, we will need an interface for the data type of expressed values. Our interface will have the entries

num-val	$: Int \to ExpVal$
bool-val	$: Bool \to ExpVal$
expval->num	$: ExpVal \to Int$
expval->bool	$: ExpVal \to Bool$

We assume that `expval->num` and `expval->bool` are undefined when given an argument that is not a number or a boolean, respectively.

3.2.3 Environments

If we are going to evaluate expressions containing variables, we will need to know the value associated with each variable. We do this by keeping those values in an *environment*, as defined in section 2.2.

An environment is a function whose domain is a finite set of variables and whose range is the denoted values. We use some abbreviations when writing about environments.

- ρ ranges over environments.
- [] denotes the empty environment.
- $[var = val]\rho$ denotes `(extend-env` var val ρ`)`.
- $[var_1 = val_1, var_2 = val_2]\rho$ abbreviates $[var_1 = val_1]([var_2 = val_2]\rho)$, etc.
- $[var_1 = val_1, var_2 = val_2, \ldots]$ denotes the environment in which the value of var_1 is val_1, etc.

We will occasionally write down complicated environments using indentation to improve readability. For example, we might write

```
[x=3]
 [y=7]
  [u=5]ρ
```

to abbreviate

```
(extend-env 'x 3
  (extend-env 'y 7
    (extend-env 'u 5 ρ)))
```

3.2.4 Specifying the Behavior of Expressions

There are six kinds of expressions in our language: one for each production with *Expression* as its left-hand side. Our interface for expressions will contain seven procedures: six constructors and one observer. We use *ExpVal* to denote the set of expressed values.

constructors:

const-exp : $Int \rightarrow Exp$
zero?-exp : $Exp \rightarrow Exp$
if-exp : $Exp \times Exp \times Exp \rightarrow Exp$
diff-exp : $Exp \times Exp \rightarrow Exp$
var-exp : $Var \rightarrow Exp$
let-exp : $Var \times Exp \times Exp \rightarrow Exp$

observer:

value-of : $Exp \times Env \rightarrow ExpVal$

Before starting on an implementation, we write down a specification for the behavior of these procedures. Following the interpreter recipe, we expect that `value-of` will look at the expression, determine what kind of expression it is, and return the appropriate value.

```
(value-of (const-exp n) ρ) = (num-val n)

(value-of (var-exp var) ρ) = (apply-env ρ var)

(value-of (diff-exp exp1 exp2) ρ)
= (num-val
    (-
      (expval->num (value-of exp1 ρ))
      (expval->num (value-of exp2 ρ))))
```

The value of a constant expression in any environment is the constant value. The value of a variable reference in an environment is determined by looking up the variable in the environment. The value of a difference expression in some environment is the difference between the value of the first operand in that environment and the value of the second operand in that environment. Of course, to be precise we have to make sure that the values of the operands are numbers, and we have to make sure that value of the result is a number represented as an expressed value.

Figure 3.3 shows how these rules work together to specify the value of an expression built by these constructors. In this and our other examples, we write «*exp*» to denote the AST for expression *exp*. We also write $\lceil n \rceil$ in place of `(num-val` *n*`)`, and $\lfloor val \rfloor$ in place of `(expval->num` *val*`)`. We will also use the fact that $\lfloor \lceil n \rceil \rfloor = n$.

Exercise 3.1 [⋆] In figure 3.3, list all the places where we used the fact that $\lfloor \lceil n \rceil \rfloor = n$.

Exercise 3.2 [⋆⋆] Give an expressed value $val \in ExpVal$ for which $\lceil \lfloor val \rfloor \rceil \neq val$.

3.2.5 Specifying the Behavior of Programs

In our language, a whole program is just an expression. In order to find the value of such an expression, we need to specify the values of the free variables in the program. So the value of a program is just the value of that expression in a suitable initial environment. We choose our initial environment to be `[i=1,v=5,x=10]`.

```
(value-of-program exp)
= (value-of exp [i=⌈1⌉,v=⌈5⌉,x=⌈10⌉])
```

3.2.6 Specifying Conditionals

The next portion of the language introduces an interface for booleans in our language. The language has one constructor of booleans, `zero?`, and one observer of booleans, the `if` expression.

The value of a `zero?` expression is a true value if and only if the value of its operand is zero. We can write this as a rule of inference like those in definition 1.1.5. We use `bool-val` as a constructor to turn a boolean into an expressed value, and `expval->num` as an extractor to check whether an expressed value is an integer, and if so, to return the integer.

Let $\rho =$ `[i=1,v=5,x=10]`.

```
(value-of
  <<-(-(x,3), -(v,i))>>
  ρ)

= ⌈(-
    ⌊(value-of <<-(x,3)>> ρ)⌋
    ⌊(value-of <<-(v,i)>> ρ)⌋)⌉

= ⌈(-
    (-
      ⌊(value-of <<x>> ρ)⌋
      ⌊(value-of <<3>> ρ)⌋)
    ⌊(value-of <<-(v,i)>> ρ)⌋)⌉

= ⌈(-
    (-
      10
      ⌊(value-of <<3>> ρ)⌋)
    (value-of <<-(v,i)>> ρ))⌉

= ⌈(-
    (-
      10
      3)
    ⌊(value-of <<-(v,i)>> ρ)⌋)⌉

= ⌈(-
    7
    ⌊(value-of <<-(v,i)>> ρ)⌋)⌉
```

```
= ⌈(-
     7
     (-
       ⌊(value-of <<v>> ρ)⌋
       ⌊(value-of <<i>> ρ)⌋))⌉

= ⌈(-
     7
     (-
       5
       ⌊(value-of <<i>> ρ)⌋))⌉

= ⌈(-
     7
     (-
       5
       1))⌉

= ⌈(-
     7
     4)⌉

= ⌈3⌉
```

Figure 3.3 A simple calculation using the specification

$$\frac{\texttt{(value-of}\ exp_1\ \rho) = val_1}{\begin{array}{l}\texttt{(value-of (zero?-exp}\ exp_1\texttt{)}\ \rho\texttt{)} \\ \quad = \begin{cases}\texttt{(bool-val \#t)} & \text{if } \texttt{(expval->num}\ val_1\texttt{)} = 0 \\ \texttt{(bool-val \#f)} & \text{if } \texttt{(expval->num}\ val_1\texttt{)} \neq 0\end{cases}\end{array}}$$

An `if` expression is an observer of boolean values. To determine the value of an `if` expression `(if-exp` exp_1 exp_2 exp_3`)`, we must first determine the value of the subexpression exp_1. If this value is a true value, the value of the entire `if-exp` should be the value of the subexpression exp_2; otherwise it should be the value of the subexpression exp_3. This is also easy to write as a rule of inference. We use `expval->bool` to extract the boolean part of an expressed value, just as we used `expval->num` in the preceding example.

$$\frac{\texttt{(value-of}\ exp_1\ \rho) = val_1}{\begin{array}{l}\texttt{(value-of (if-exp}\ exp_1\ exp_2\ exp_3\texttt{)}\ \rho\texttt{)} \\ \quad = \begin{cases}\texttt{(value-of}\ exp_2\ \rho\texttt{)} & \text{if } \texttt{(expval->bool}\ val_1\texttt{)} = \texttt{\#t} \\ \texttt{(value-of}\ exp_3\ \rho\texttt{)} & \text{if } \texttt{(expval->bool}\ val_1\texttt{)} = \texttt{\#f}\end{cases}\end{array}}$$

Rules of inference like this make the intended behavior of any individual expression easy to specify, but they are not very good for displaying a deduction. An antecedent like `(value-of` exp_1 ρ`)` $= val_1$ denotes a subcomputation, so a calculation should be a tree, much like the one on page 5. Unfortunately, such trees can be difficult to read. We therefore often recast our rules as equations. We can then use substitution of equals for equals to display a calculation.

For an `if-exp` , the equational specification is

```
(value-of (if-exp exp1 exp2 exp3) ρ)
= (if (expval->bool (value-of exp1 ρ))
    (value-of exp2 ρ)
    (value-of exp3 ρ))
```

Figure 3.4 shows a simple calculation using these rules.

3.2.7 Specifying `let`

Next we address the problem of creating new variable bindings with a `let` expression. We add to the interpreted language a syntax in which the keyword `let` is followed by a declaration, the keyword `in`, and the body. For example,

Let $\rho = [\mathtt{x}=\lceil 33 \rceil, \mathtt{y}=\lceil 22 \rceil]$.

```
(value-of
  <<if zero?(-(x,11)) then -(y,2) else -(y,4)>>
  ρ)

= (if (expval->bool (value-of <<zero?(-(x,11))>> ρ))
    (value-of <<-(y,2)>> ρ)
    (value-of <<-(y,4)>> ρ))

= (if (expval->bool (bool-val #f))
    (value-of <<-(y,2)>> ρ)
    (value-of <<-(y,4)>> ρ))

= (if #f
    (value-of <<-(y,2)>> ρ)
    (value-of <<-(y,4)>> ρ))

= (value-of <<-(y,4)>> ρ)

= ⌈18⌉
```

Figure 3.4 A simple calculation for a conditional expression

```
let x = 5
in -(x,3)
```

The `let` variable is bound in the body, much as a `lambda` variable is bound (see section 1.2.4).

The entire `let` form is an expression, as is its body, so `let` expressions may be nested, as in

```
let z = 5
in let x = 3
   in let y = -(x,1)     % here x = 3
      in let x = 4
         in -(z, -(x,y)) % here x = 4
```

In this example, the reference to `x` in the first difference expression refers to the outer declaration, whereas the reference to `x` in the other difference expression refers to the inner declaration, and thus the entire expression's value is 3.

The right-hand side of the `let` is also an expression, so it can be arbitrarily complex. For example,

```
let x = 7
in let y = 2
   in let y = let x = -(x,1)
              in -(x,y)
      in -(-(x,8), y)
```

Here the `x` declared on the third line is bound to 6, so the value of `y` is 4, and the value of the entire expression is $((-1) - 4) = -5$.

We can write down the specification as a rule.

$$\frac{\texttt{(value-of}\ exp_1\ \rho) = val_1}{\begin{array}{l}\texttt{(value-of (let-exp}\ var\ exp_1\ body)\ \rho\texttt{)} \\ \quad = \texttt{(value-of}\ body\ [var = val_1]\rho\texttt{)}\end{array}}$$

As before, it is often more convenient to recast this as the equation

```
(value-of (let-exp var exp1 body) ρ)
= (value-of body [var=(value-of exp1 ρ)]ρ)
```

Figure 3.5 shows an example. There ρ_0 denotes an arbitrary environment.

3.2.8 Implementing the Specification of LET

Our next task is to implement this specification as a set of Scheme procedures. Our implementation uses SLLGEN as a front end, which means that expressions will be represented by a data type like the one in figure 3.6. The representation of expressed values in our implementation is shown in figure 3.7. The data type declares the constructors `num-val` and `bool-val` for converting integers and booleans to expressed values. We also define extractors for converting from an expressed value back to either an integer or a boolean. The extractors report an error if an expressed value is not of the expected kind.

```
(value-of
  <<let x = 7
    in let y = 2
       in let y = let x = -(x,1) in -(x,y)
          in -(-(x,8),y)>>
  ρ0)

= (value-of
    <<let y = 2
      in let y = let x = -(x,1) in -(x,y)
         in -(-(x,8),y)>>
    [x=⌈7⌉]ρ0)

= (value-of
    <<let y = let x = -(x,1) in -(x,y)
      in -(-(x,8),y)>>
    [y=⌈2⌉][x=⌈7⌉]ρ0)
```

Let ρ_1 = `[y=⌈2⌉][x=⌈7⌉]`ρ_0.

```
= (value-of
    <<-(-(x,8),y)>>
    [y=(value-of <<let x = -(x,1) in -(x,y)>> ρ1)]
     ρ1)

= (value-of
    <<-(-(x,8),y)>>
    [y=(value-of <<-(x,2)>> [x=(value-of <<-(x,1)>> ρ1)]ρ1)]
     ρ1)

= (value-of
    <<-(-(x,8),y)>>
    [y=(value-of <<-(x,2)>> [x=⌈6⌉]ρ1)]
     ρ1)

= (value-of
    <<-(-(x,8),y)>>
    [y=⌈4⌉]ρ1)

= ⌈(- (- 7 8) 4)⌉

= ⌈-5⌉
```

Figure 3.5 An example of `let`

```
(define-datatype program program?
  (a-program
    (exp1 expression?)))

(define-datatype expression expression?
  (const-exp
    (num number?))
  (diff-exp
    (exp1 expression?)
    (exp2 expression?))
  (zero?-exp
    (exp1 expression?))
  (if-exp
    (exp1 expression?)
    (exp2 expression?)
    (exp3 expression?))
  (var-exp
    (var identifier?))
  (let-exp
    (var identifier?)
    (exp1 expression?)
    (body expression?)))
```

Figure 3.6 Syntax data types for the LET language

We can use any implementation of environments, provided that it meets the specification in section 2.2. The procedure `init-env` constructs the specified initial environment used by `value-of-program`.

init-env : () → *Env*
usage: (init-env) = [i=⌈1⌉,v=⌈5⌉,x=⌈10⌉]

```
(define init-env
  (lambda ()
    (extend-env
     'i (num-val 1)
     (extend-env
      'v (num-val 5)
      (extend-env
       'x (num-val 10)
       (empty-env))))))
```

```
(define-datatype expval expval?
  (num-val
    (num number?))
  (bool-val
    (bool boolean?)))

expval->num : ExpVal → Int
(define expval->num
  (lambda (val)
    (cases expval val
      (num-val (num) num)
      (else (report-expval-extractor-error 'num val)))))

expval->bool : ExpVal → Bool
(define expval->bool
  (lambda (val)
    (cases expval val
      (bool-val (bool) bool)
      (else (report-expval-extractor-error 'bool val)))))
```

Figure 3.7 Expressed values for the LET language

Now we can write down the interpreter, shown in figures 3.8 and 3.9. The main procedure is `run`, which takes a string, parses it, and hands the result to `value-of-program`. The most interesting procedure is `value-of`, which takes an expression and an environment and uses the interpreter recipe to calculate the answer required by the specification. In the listing below we have inserted the relevant specification rules to show how the code for `value-of` comes from the specification.

In the following exercises, and throughout the book, the phrase "extend the language by adding ..." means to write down additional rules or equations to the language specification, and to implement the feature by adding or modifying the associated interpreter.

Exercise 3.3 [⋆] Why is subtraction a better choice than addition for our single arithmetic operation?

Exercise 3.4 [⋆] Write out the derivation of figure 3.4 as a derivation tree in the style of the one on page 5.

```
run : String → ExpVal
(define run
  (lambda (string)
    (value-of-program (scan&parse string))))

value-of-program : Program → ExpVal
(define value-of-program
  (lambda (pgm)
    (cases program pgm
      (a-program (exp1)
        (value-of exp1 (init-env))))))

value-of : Exp × Env → ExpVal
(define value-of
  (lambda (exp env)
    (cases expression exp

      (value-of (const-exp n) ρ) = n
      (const-exp (num) (num-val num))

      (value-of (var-exp var) ρ) = (apply-env ρ var)
      (var-exp (var) (apply-env env var))

      (value-of (diff-exp exp1 exp2) ρ) =
        ⌈(- ⌊(value-of exp1 ρ)⌋ ⌊(value-of exp2 ρ)⌋)⌉
      (diff-exp (exp1 exp2)
        (let ((val1 (value-of exp1 env))
              (val2 (value-of exp2 env)))
          (let ((num1 (expval->num val1))
                (num2 (expval->num val2)))
            (num-val
              (- num1 num2)))))))
```

Figure 3.8 Interpreter for the LET language

$$\frac{\texttt{(value-of } exp_1 \ \rho) = val_1}{\texttt{(value-of (zero?-exp } exp_1\texttt{) } \rho\texttt{)} = \begin{cases}\texttt{(bool-val \#t)} & \text{if } \texttt{(expval->num } val_1\texttt{)} = 0\\ \texttt{(bool-val \#f)} & \text{if } \texttt{(expval->num } val_1\texttt{)} \neq 0\end{cases}}$$

```
(zero?-exp (exp1)
  (let ((val1 (value-of exp1 env)))
    (let ((num1 (expval->num val1)))
      (if (zero? num1)
        (bool-val #t)
        (bool-val #f)))))
```

$$\frac{\texttt{(value-of } exp_1 \ \rho) = val_1}{\texttt{(value-of (if-exp } exp_1 \ exp_2 \ exp_3\texttt{) } \rho\texttt{)} = \begin{cases}\texttt{(value-of } exp_2 \ \rho\texttt{)} & \text{if } \texttt{(expval->bool } val_1\texttt{)} = \texttt{\#t}\\ \texttt{(value-of } exp_3 \ \rho\texttt{)} & \text{if } \texttt{(expval->bool } val_1\texttt{)} = \texttt{\#f}\end{cases}}$$

```
(if-exp (exp1 exp2 exp3)
  (let ((val1 (value-of exp1 env)))
    (if (expval->bool val1)
      (value-of exp2 env)
      (value-of exp3 env))))
```

$$\frac{\texttt{(value-of } exp_1 \ \rho) = val_1}{\texttt{(value-of (let-exp } var \ exp_1 \ body\texttt{) } \rho\texttt{)} = \texttt{(value-of } body \ [var = val_1]\rho\texttt{)}}$$

```
(let-exp (var exp1 body)
  (let ((val1 (value-of exp1 env)))
    (value-of body
      (extend-env var val1 env))))))
```

Figure 3.9 Interpreter for the LET language, continued

Exercise 3.5 [⋆] Write out the derivation of figure 3.5 as a derivation tree in the style of the one on page 5.

Exercise 3.6 [⋆] Extend the language by adding a new operator `minus` that takes one argument, n, and returns $-n$. For example, the value of `minus(-(minus(5),9))` should be 14.

Exercise 3.7 [⋆] Extend the language by adding operators for addition, multiplication, and integer quotient.

Exercise 3.8 [⋆] Add a numeric equality predicate `equal?` and numeric order predicates `greater?` and `less?` to the set of operations in the defined language.

Exercise 3.9 [⋆⋆] Add list processing operations to the language, including `cons`, `car`, `cdr`, `null?` and `emptylist`. A list should be able to contain any expressed value, including another list. Give the definitions of the expressed and denoted values of the language, as in section 3.2.2. For example,

```
let x = 4
in cons(x,
        cons(cons(-(x,1),
                  emptylist),
             emptylist))
```

should return an expressed value that represents the list `(4 (3))`.

Exercise 3.10 [⋆⋆] Add an operation `list` to the language. This operation should take any number of arguments, and return an expressed value containing the list of their values. For example,

```
let x = 4
in list(x, -(x,1), -(x,3))
```

should return an expressed value that represents the list `(4 3 1)`.

Exercise 3.11 [⋆] In a real language, one might have many operators such as those in the preceding exercises. Rearrange the code in the interpreter so that it is easy to add new operators.

Exercise 3.12 [⋆] Add to the defined language a facility that adds a `cond` expression. Use the grammar

$$\textit{Expression} ::= \texttt{cond}\ \{\textit{Expression}\ \texttt{==>}\ \textit{Expression}\}^*\ \texttt{end}$$

In this expression, the expressions on the left-hand sides of the `==>`'s are evaluated in order until one of them returns a true value. Then the value of the entire expression is the value of the corresponding right-hand expression. If none of the tests succeeds, the expression should report an error.

Exercise 3.13 [⋆] Change the values of the language so that integers are the only expressed values. Modify `if` so that the value 0 is treated as false and all other values are treated as true. Modify the predicates accordingly.

Exercise 3.14 [⋆⋆] As an alternative to the preceding exercise, add a new nonterminal *Bool-exp* of boolean expressions to the language. Change the production for conditional expressions to say

$$\textit{Expression} ::= \texttt{if}\ \textit{Bool-exp}\ \texttt{then}\ \textit{Expression}\ \texttt{else}\ \textit{Expression}$$

Write suitable productions for *Bool-exp* and implement `value-of-bool-exp`. Where do the predicates of exercise 3.8 wind up in this organization?

Exercise 3.15 [⋆] Extend the language by adding a new operation `print` that takes one argument, prints it, and returns the integer 1. Why is this operation not expressible in our specification framework?

Exercise 3.16 [⋆⋆] Extend the language so that a `let` declaration can declare an arbitrary number of variables, using the grammar

$$\textit{Expression} ::= \texttt{let}\ \{\textit{Identifier} = \textit{Expression}\}^*\ \texttt{in}\ \textit{Expression}$$

As in Scheme's `let`, each of the right-hand sides is evaluated in the current environment, and the body is evaluated with each new variable bound to the value of its associated right-hand side. For example,

```
let x = 30
in let x = -(x,1)
       y = -(x,2)
   in -(x,y)
```

should evaluate to 1.

Exercise 3.17 [⋆⋆] Extend the language with a `let*` expression that works like Scheme's `let*`, so that

```
let x = 30
in let* x = -(x,1) y = -(x,2)
   in -(x,y)
```

should evaluate to 2.

Exercise 3.18 [⋆⋆] Add an expression to the defined language:

$$\textit{Expression} ::= \texttt{unpack}\ \{\textit{Identifier}\}^*\ \texttt{=}\ \textit{Expression}\ \texttt{in}\ \textit{Expression}$$

so that `unpack x y z = lst in ...` binds `x`, `y`, and `z` to the elements of `lst` if `lst` is a list of exactly three elements, and reports an error otherwise. For example, the value of

```
let u = 7
in unpack x y = cons(u,cons(3,emptylist))
   in -(x,y)
```

should be 4.

3.3 PROC: A Language with Procedures

So far our language has only the operations that were included in the original language. For our interpreted language to be at all useful, we must allow new procedures to be created. We call the new language PROC.

We will follow the design of Scheme, and let procedures be expressed values in our language, so that

$$ExpVal = Int + Bool + Proc$$
$$DenVal = Int + Bool + Proc$$

where *Proc* is a set of values representing procedures. We will think of *Proc* as an abstract data type. We consider its interface and specification below.

We will also need syntax for procedure creation and calling. This is given by the productions

Expression ::= `proc (`*Identifier*`)` *Expression*
`proc-exp (var body)`

Expression ::= `(`*Expression* *Expression*`)`
`call-exp (rator rand)`

In `(proc-exp` *var* *body*`)`, the variable *var* is the *bound variable* or *formal parameter*. In a procedure call `(call-exp` exp_1 exp_2`)`, the expression exp_1 is the *operator* and exp_2 is the *operand* or *actual parameter*. We use the word *argument* to refer to the value of an actual parameter.

Here are two simple programs in this language.

```
let f = proc (x) -(x,11)
in (f (f 77))

(proc (f) (f (f 77))
 proc (x) -(x,11))
```

The first program creates a procedure that subtracts 11 from its argument. It calls the resulting procedure `f`, and then applies `f` twice to 77, yielding the answer 55. The second program creates a procedure that takes its argument and applies it twice to 77. The program then applies this procedure to the subtract-11 procedure. The result is again 55.

We now turn to the data type *Proc*. Its interface consists of the constructor `procedure`, which tells how to build a procedure value, and the observer `apply-procedure`, which tells how to apply a procedure value.

Our next task is to determine what information must be included in a value representing a procedure. To do this, we consider what happens when we write a `proc` expression in an arbitrary position in our program.

The lexical scope rule tells us that when a procedure is applied, its body is evaluated in an environment that binds the formal parameter of the procedure to the argument of the call. Variables occurring free in the procedure should also obey the lexical binding rule. Consider the expression

```
let x = 200
in let f = proc (z) -(z,x)
   in let x = 100
      in let g = proc (z) -(z,x)
         in -((f 1), (g 1))
```

Here we evaluate the expression `proc (z) -(z,x)` twice. The first time we do it, `x` is bound to 200, so by the lexical scope rule, the result is a procedure that subtracts 200 from its argument. We name this procedure `f`. The second time we do it, `x` is bound to 100, so the resulting procedure should subtract 100 from its argument. We name this procedure `g`.

These two procedures, created from identical expressions, must behave differently. We conclude that the value of a `proc` expression must depend in some way on the environment in which it is evaluated. Therefore the constructor `procedure` must take three arguments: the bound variable, the body, and the environment. The specification for a `proc` expression is

```
(value-of (proc-exp var body) ρ)
= (proc-val (procedure var body ρ))
```

where `proc-val` is a constructor, like `bool-val` or `num-val`, that builds an expressed value from a *Proc*.

At a procedure call, we want to find the value of the operator and the operand. If the value of the operator is a `proc-val`, then we want to apply it to the value of the operand.

```
(value-of (call-exp rator rand) ρ)
= (let ((proc (expval->proc (value-of rator ρ)))
        (arg (value-of rand ρ)))
    (apply-procedure proc arg))
```

Here we rely on a tester `expval->proc`, like `expval->num`, to test whether the value of `(value-of` *rator* `ρ)`, an expressed value, was constructed by `proc-val`, and if so to extract the underlying procedure.

Last, we consider what happens when `apply-procedure` is invoked. As we have seen, the lexical scope rule tells us that when a procedure is applied, its body is evaluated in an environment that binds the formal parameter of the procedure to the argument of the call. Furthermore any other variables must have the same values they had at procedure-creation time. Therefore these procedures should satisfy the condition

```
(apply-procedure (procedure var body ρ) val)
= (value-of body [var=val]ρ)
```

3.3.1 An Example

Let's do an example to show how the pieces of the specification fit together. This is a calculation using the *specification,* not the implementation, since we have not yet written down the implementation of procedures. Let ρ be any environment.

```
(value-of
  <<let x = 200
    in let f = proc (z) -(z,x)
       in let x = 100
          in let g = proc (z) -(z,x)
             in -((f 1), (g 1))>>
  ρ)

= (value-of
    <<let f = proc (z) -(z,x)
      in let x = 100
         in let g = proc (z) -(z,x)
            in -((f 1), (g 1))>>
    [x=⌈200⌉]ρ)


= (value-of
    <<let x = 100
      in let g = proc (z) -(z,x)
         in -((f 1), (g 1))>>
    [f=(proc-val (procedure z <<-(z,x)>> [x=⌈200⌉]ρ))]
     [x=⌈200⌉]ρ)

= (value-of
    <<let g = proc (z) -(z,x)
      in -((f 1), (g 1))>>
    [x=⌈100⌉]
     [f=(proc-val (procedure z <<-(z,x)>> [x=⌈200⌉]ρ))]
      [x=⌈200⌉]ρ)
```